GitHub Copilot Orchestration Framework

A reusable multi-agent orchestration setup for GitHub Copilot

Generated 2026-05-02 (rev 2 — pre-flight safety)

Cover

This document is the printable version of the **GitHub Copilot Orchestration Framework** — a tech-stack-agnostic multi-agent setup grounded in GitHub Copilot's documented customization features.

The framework lives at ~/Desktop/github-copilot-orchestration-framework/ (local) and on GitHub as a private repository.

What's inside (in this order):

1. **README** — purpose + quick-start for new and existing projects
2. **Docs** (9 chapters) — principles, architecture, agents, handoff schema, instructions+prompts, invocation modes, folder structure, common pitfalls (now 18), runbook
3. **Prompts** (3) — inventory, bootstrap (now with mandatory pre-flight safety

checks), refinement
4. **Templates** (11) — drop-in agent files, instructions, prompts, chatmode, schema, INDEX, spoonfeeder, archive README, copilot-instructions

Revision 2 changes: the BOOTSTRAP-PROMPT now includes mandatory pre-flight safety checks (snapshot, naming-collision detection, applyTo glob conflict detection, drift detection, decision gate) so the framework safely coexists with existing Copilot configuration. New Pitfall 17 documents the bootstrap-on-existing-config scenario.

The most actionable single chapter is **Chapter 9 — Runbook**.

README

GitHub Copilot Orchestration Framework

> **Purpose.** A reusable multi-agent orchestration setup for GitHub Copilot that prevents cascading hallucinations, enforces evidence-based handoffs between Copilot custom agents, and makes Copilot usable on production codebases by teams. Tech-stack agnostic — drops into any project (web, mobile, backend, ML, infra) in 2-4 hours.

This framework uses GitHub Copilot's **own** customization surface — `.github/copilot-instructions.md`, `.github/instructions/`, `.github/prompts/`, `.github/agents/`, `.github/chatmodes/` — and adds documentation conventions on top. No external tools, no extensions, no Marketplace dependencies.

Who this is for

- Engineering teams using GitHub Copilot on real production codebases
- Projects with multiple roles, multiple data systems, or compliance/security/payments concerns
- Anyone who has experienced Copilot drifting on rules, hallucinating cross-file claims, or producing inconsistent suggestions across team members
- Teams who use Copilot in a mix of: VS Code Chat, JetBrains, Visual Studio, Copilot CLI, GitHub.com Chat, and the Copilot cloud agent (autonomous coding agent)

If your project is a solo prototype or under ~5,000 lines, you probably don't need this yet — default Copilot is enough. Adopt this when complexity passes the threshold.

The 30-second pitch

Default Copilot is excellent for inline suggestions and quick chat. For complex projects, you need:

- **One orchestrator custom agent that delegates to specialist custom agents** (instead of asking one chat session to do everything and flooding context)
- **Bidirectional structured handoffs with evidence-bound claims** (instead of free-form prompts that propagate hallucinations across hops)
- **Per-agent runtime constraints** — REVIEW-ONLY agents physically can't write; tool allowlists scope MCP access; `target` field scopes agents to specific environments
- **A `failure_condition` field** that forces specialist agents to STOP if the orchestrator's premise turns out wrong (instead of pushing through on a false hypothesis)
- **Three-tier docs** — orientation maps for Copilot, canonical refs for humans, frozen archive for history
- **Path-globbed instruction files** — `.github/instructions/<NAME>.instructions.md` with `applyTo:` glob (auto-loaded when the matching file is being edited)

This framework gives you all of that using Copilot's documented features. No runtime hooks, no external dependencies — just markdown + Copilot's built-in customization mechanisms.

Copilot customization surfaces this framework uses

Mechanism	File	Loaded when	Best for
Repository-wide instructions	.github/copilot-instructions.md	Every repository request	Project-wide router (golden rules + workflow + agent map)
Path-specific instructions	.github/instructions/<NAME>.instructions.md (with applyTo: glob)	Editing files matching the glob	Domain-scoped invariants ("rules")
Prompt files	.github/prompts/<NAME>.prompt.md	Manually invoked via /<name> in Chat	Repeatable workflows ("skills")
Custom agents	.github/agents/<NAME>.md	Selected manually OR auto-routed	Orchestrator + specialists
Custom chat modes	.github/chatmodes/<NAME>.chatmode.md	Selected from Chat dropdown	Personas (optional, secondary to agents)

All five mechanisms are documented Copilot features. See docs/02-ARCHITECTURE.md for which to use when.

What's in the box

```
github-copilot-orchestration-framework/
├── README.md                          ← this file
├── GitHub-Copilot-Orchestration-Framework.pdf   ← consolidated printable
├── LICENSE
│
├── docs/                              ← framework documentation (9 chapters)
│   ├── 01-PRINCIPLES.md               ← seven core principles
│   ├── 02-ARCHITECTURE.md             ← .github/ layout (instructions / prompts / agents / chatmodes)
│   ├── 03-AGENTS-GUIDE.md             ← how to design orchestrator + specialists for Copilot
│   ├── 04-HANDOFF-SCHEMA.md           ← bidirectional schema + worked examples
│   ├── 05-INSTRUCTIONS-AND-PROMPTS.md ← path-globbed instructions + prompt files
│   ├── 06-INVOCATION-MODES.md         ← Chat vs Edit vs Cloud Agent vs CLI
│   ├── 07-FOLDER-STRUCTURE.md         ← three-tier doc organization
│   ├── 08-COMMON-PITFALLS.md          ← Copilot-specific lessons + framework lessons
│   └── 09-RUNBOOK.md                  ← step-by-step bootstrap (~2-4 hours)
│
├── prompts/                           ← ready-to-paste Chat prompts for bootstrapping
│   ├── INVENTORY-PROMPT.md
│   ├── BOOTSTRAP-PROMPT.md
│   └── REFINEMENT-PROMPT.md
│
└── templates/                         ← drop-in templates with placeholders
    ├── orchestrator-agent.md.template     ← .github/agents/<project>-orchestrator.md
    ├── specialist-agent.md.template       ← .github/agents/<specialist>.md
    ├── review-only-agent.md.template      ← REVIEW-ONLY specialist
    ├── HANDOFF_SCHEMA.md.template         ← docs/ai-context/HANDOFF_SCHEMA.md
    ├── INDEX.md.template                  ← docs/ai-context/INDEX.md
    ├── SPOONFEEDER.md.template            ← docs/ai-context/ORCHESTRATION_SPOONFEEDER.md
    ├── copilot-instructions.md.template   ← .github/copilot-instructions.md (root router)
    ├── instructions.md.template           ← .github/instructions/<name>.instructions.md
    ├── prompt.md.template                 ← .github/prompts/<name>.prompt.md
    ├── chatmode.md.template               ← .github/chatmodes/<name>.chatmode.md
    └── archive-README.md.template
```

Quick start — pick your scenario

Scenario A — Brand new project (greenfield)

```
# 1. Verify Copilot is installed in your IDE (VS Code / JetBrains / Visual Studio)
#    and you're signed in to a GitHub account with Copilot access

# 2. Open the new project in your IDE
cd ~/path/to/new-project

# 3. Open Copilot Chat (Ctrl+Cmd+I in VS Code)
#    Paste prompts/BOOTSTRAP-PROMPT.md
#    Replace <framework path> with the absolute path to this repo on your machine
```

For greenfield, you can skip the inventory step and tell Copilot what specialists you

want directly:

```
Set up the GitHub Copilot Orchestration Framework for a new
<Next.js + Postgres + Stripe> project named "<your-project-name>".

Specialists I want (each becomes .github/agents/<name>.md):
  - <project-slug>-orchestrator
  - frontend-ui
  - backend-api
  - database
  - payments
  - auth-security
  - qa-functional
  - release-devops
  - security-privacy (REVIEW-ONLY)

Use templates from <absolute path to this repo>/templates/.
Show me each generated file before saving.
```

Estimated time: **45 min - 1 hour**.

Scenario B — Existing project (brownfield)

⚠ **If your project ALREADY has Copilot configuration** (e.g. you ran VS Code's /init, or your team has been adding to .github/copilot-instructions.md for months), the bootstrap will detect this and ask before overwriting. The framework includes mandatory pre-flight safety checks:

- **Pre-flight 1** — auto-snapshot existing config to .github-pre-bootstrap-backup/
- **Pre-flight 2** — naming-collision check (per file — STOP for explicit user decision)
- **Pre-flight 3** — applyTo: glob conflict check
- **Pre-flight 4** — drift detection on existing .github/copilot-instructions.md
- **Pre-flight 5** — existing chatmode/agent style detection
- **Decision gate** — STOP if any pre-flight raised a <NEEDS USER CONFIRMATION> flag

You can also do an extra manual snapshot before starting:

```
mkdir -p .github-pre-bootstrap-backup
cp -r .github .github-pre-bootstrap-backup/ 2>/dev/null
[ -f AGENTS.md ] && cp AGENTS.md .github-pre-bootstrap-backup/
```

The framework's BOOTSTRAP-PROMPT will repeat the snapshot inside the prompt — this is just belt-and-suspenders.

```
# 1. Verify Copilot is installed and you're signed in

# 2. Open the existing project in your IDE
cd ~/path/to/existing-project

# 3. Make sure git is clean
git status
git checkout -b setup/copilot-orchestration

# 4. Open Copilot Chat
#    Paste prompts/INVENTORY-PROMPT.md
#    Replace <framework path> with the absolute path to this repo on your machine

# 5. Review/adjust Copilot's proposed inventory + answer Open Questions
#    INVENTORY scans for existing .github/agents/, /instructions/, /prompts/, /chatmodes/, AGENT

# 6. Paste prompts/BOOTSTRAP-PROMPT.md (in the same Chat session)
#    BOOTSTRAP runs pre-flight safety checks BEFORE creating any files
#    If existing config is detected, you'll be asked to confirm per-file decisions

# 7. Verify in terminal
ls .github/agents/              # should list your project specialists
ls .github/instructions/        # should list your path-globbed instructions
ls .github/prompts/             # should list your prompt files
diff .github-pre-bootstrap-backup/copilot-instructions.md .github/copilot-instructions.md  # if existed

# 8. Test in Chat — pick a real task
#    In VS Code Chat: select your orchestrator agent from the agent picker
#    Or: type @<project-slug>-orchestrator <your task>

# 9. Commit and PR (note: .github-pre-bootstrap-backup/ is gitignored)
git add .github/ docs/ai-context/ docs/_archive/ AGENTS.md .gitignore
git commit -m "chore: bootstrap GitHub Copilot orchestration framework"
git push -u origin setup/copilot-orchestration
gh pr create --base <your-base-branch> --title "Bootstrap Copilot orchestration framework"
```

Estimated time: **2-4 hours** (split across phases — see docs/09-RUNBOOK.md).

Scenario C — Just want to read the framework

Read the **PDF** (GitHub-Copilot-Orchestration-Framework.pdf). Or browse the markdown files in docs/ for clickable cross-links.

The most actionable single chapter is **docs/09-RUNBOOK.md**.

Prerequisites

- **GitHub Copilot subscription** (Individual / Business / Enterprise)
- **Copilot extension** installed in your IDE (VS Code, Visual Studio, JetBrains, Eclipse, or Xcode)
- **Custom agents support** — verify in your IDE settings (some features require recent Copilot extension versions)
- **Git** + optional **GitHub CLI** (gh) for PR creation

Note: prompt files and custom agents are currently in **public preview** in some IDEs. Functionality may evolve. Check the Copilot release notes when adopting new features.

What this framework does NOT include

- **Runtime hooks.** Copilot doesn't expose pre-tool-use hooks the way some AI agents do. Documentation enforcement is the framework's primary discipline layer.
- **maxTurns enforcement.** Copilot agents don't currently support a maxTurns field. The orchestrator's "soft hop limit" is documentation-only.
- **Code generation.** This is purely a documentation + agent-config framework. Your application code stays untouched.
- **Vendor lock-in.** No external services, no SaaS, no API keys beyond your existing Copilot subscription.

Customizing the framework for your project

Three things change per project:

1. **Project name + slug** (used to name the orchestrator: <slug>-orchestrator)
2. **Specialist list** (5-10 specialists matching your project's domain boundaries)
3. **applyTo: globs in path-specific instructions** (matched to your actual code paths)

Everything else (the handoff schema, the universal evidence rule, the failure_condition pattern, the three-tier docs structure) stays the same across projects.

See **docs/03-AGENTS-GUIDE.md** for how to pick your specialist list and **docs/05-INSTRUCTIONS-AND-PROMPTS.md** for how to write path-globbed instructions and prompt files.

Maintenance

After bootstrapping, the framework needs minimal upkeep:

- **Per task:** instruction files accumulate naturally as production teaches you new gotchas
- **Per quarter:** run prompts/REFINEMENT-PROMPT.md to audit specialist scope drift, archive stale docs
- **Per major refactor:** the context-librarian specialist (if you spawned one) handles docs reorganization

How this differs from the Claude Code version

If you've seen the Claude Code Orchestration Framework, the patterns are similar but the implementation differs because Copilot's customization model differs:

Concept	Claude Code	GitHub Copilot
Project router	CLAUDE.md	.github/copilot-instructions.md (or AGENTS.md for cross-AI)
Specialists	.claude/agents/<name>.md	.github/agents/<NAME>.md
Path-globbed rules	.claude/rules/<name>.md (with applies_to: in body)	.github/instructions/<NAME>.instructions.md (with applyTo: in frontmatter — direct equivalent, cleaner)
Repeatable workflows	.claude/skills/<name>/SKILL.md	.github/prompts/<NAME>.prompt.md (invoked via /<name>)
Personas (optional)	n/a	.github/chatmodes/<NAME>.chatmode.md
Tool allowlists	tools: field	tools: field (same syntax)
MCP scoping	mcpServers: field	mcp-servers: field (note hyphen, not camelCase)
Cross-agent invocation	Agent(name1, name2) allowlist	agent tool alias + cross-references in body
maxTurns	Supported	NOT supported (documentation discipline only)
Subagent context isolation	Strong (separate context windows)	Varies by surface — see docs/06-INVOCATION-MODES.md
Invocation modes	claude / claude --agent <name>	IDE Chat / Cloud Agent / CLI / Code Review (4+ surfaces)

Both frameworks share: the bidirectional handoff schema, the universal evidence rule, the failure_condition pattern, three-tier docs, and the principle of documentation-discipline-over-runtime-enforcement.

Frequently asked

Q: Do custom agents work in inline completions? No. Custom agents apply to Chat, Cloud Agent, and (depending on IDE) Edits. Inline completions use only repository-wide instructions (.github/copilot-instructions.md) and path-specific instructions (.github/instructions/).

Q: My team uses both VS Code and JetBrains. Will this framework work for both? Yes — the .github/ files are shared across IDEs. Some features (like prompt files) work identically in VS Code, Visual Studio, JetBrains. The cloud agent on github.com uses the same files plus target: field scoping if needed.

Q: What's the difference between custom agents and custom chat modes? Custom agents (.github/agents/<NAME>.md) are the newer format with full metadata (tools, mcp-servers, model, target, user-invocable). Custom chat modes (.github/chatmodes/<NAME>.chatmode.md) are the older format with similar but more limited fields. The framework recommends agents over chat modes for new projects — .agent.md is the more future-proof format.

Q: Will this conflict with my existing .github/copilot-instructions.md? The framework will REPLACE your existing file. Back it up first if it has content you want to preserve. The new router includes a section for project-specific instructions you can keep.

Q: Can I share this with another engineer who isn't on my team? This repo is private. Add them as a collaborator on GitHub, or clone the framework folder and share. The framework itself is MIT-licensed.

License

MIT (see LICENSE). Use freely. Adapt freely. Attribution appreciated but not required.

Sources

This framework's design is grounded in the official GitHub Copilot documentation: - About customizing GitHub Copilot Chat responses - Adding repository custom instructions - Custom agents configuration - About custom agents (cloud agent) - Prompt files tutorial - Creating custom agents for Copilot cloud agent

ewpage

01 — Principles

The seven principles this framework rests on. Every other doc, prompt, and

template implements these.

1. Orchestrator-worker pattern, documented not coded

One coordinator agent reads tasks, classifies them, picks the right specialists, and aggregates findings. Specialists do focused work in their own context (where the surface allows isolation).

The orchestration is **prescriptive documentation**, not runtime code. The "orchestrator" is a .github/agents/<project>-orchestrator.md file with instructions. There is no orchestration service, no message queue, no central process.

This matters because: - It works with any Copilot surface that recognizes custom agents (Chat in VS Code / JetBrains / Visual Studio, the cloud agent on github.com, the Copilot CLI) - It's fully transparent — every rule is in version control as plain markdown - It's reversible — delete .github/agents/, .github/instructions/, .github/prompts/ and you're back to default Copilot - Hard runtime enforcement (when Copilot eventually exposes hooks) can be added later as a separate hardening pass

2. Each specialist runs with focused tool scope

Each specialist agent declares a tools: array in its frontmatter. Copilot enforces this at runtime — the agent physically cannot use tools not listed.

For specialists that need MCP server access (e.g. browser testing, database introspection), declare them in the mcp-servers: field with explicit scope.

Examples: - A legal-compliance specialist gets tools: ['codebase', 'search', 'usages'] — no edit tools at all - A frontend-ui specialist gets edit tools plus a Chrome DevTools MCP for verification - A backend-api specialist gets edit tools plus a database MCP for schema introspection

This is the **single most important runtime defense** the framework provides. A REVIEW-ONLY specialist cannot hallucinate code into your repo because the harness blocks the tool call.

> **Note on context isolation:** Custom agents in Copilot Chat run within the active Chat session's context (unlike Claude Code subagents which run in fully isolated context windows). The cloud agent runs in true isolation. See docs/06-INVOCATION-MODES.md for the practical implications.

3. Universal evidence rule — "no evidence, no claim"

If a claim cannot be tied to a file:line, table:column, rule, doc anchor, test, or command output, it must be marked as an assumption (confidence: low outbound, or moved to unverified_claims inbound) and cannot be passed downstream as fact.

This rule appears verbatim in: - The handoff schema (both directions) - The orchestrator's "outbound discipline" - Every specialist's "incoming handoff validation" - Every path-globbed instruction file's preamble

It is the **core defense against cascading hallucinations** across hops. An orchestrator that hallucinates "the bug is in foo.ts:42" must cite that path; the specialist re-verifies before editing; if wrong, the specialist returns status: claim_rejected with the evidence of what it actually found.

4. Failure condition — articulate what would prove you wrong

Every outbound handoff includes a failure_condition field: one sentence stating what observable evidence would prove the orchestrator's hypothesis or delegation premise wrong. If the specialist observes that condition, it stops, returns claim_rejected, and includes the triggering evidence.

This is the **inverse of verify_before_acting**. Where verify_before_acting says "check these before acting," failure_condition says "if you observe this, STOP." Together they bracket the work in falsifiable claims.

Forcing the orchestrator to articulate failure_condition is itself a thinking discipline — if you can't name what would falsify your hypothesis, you don't have a hypothesis, you have a guess.

5. Tools are runtime-enforced; everything else is documentation discipline

Copilot's harness enforces: - tools: allowlists (specialist physically cannot use tools not listed in the agent file) - disable-model-invocation: (prevents auto-routing to this agent) - user-invocable: (controls manual selection) - target: (scopes agent to specific environments — vscode or github-copilot) - mcp-servers: per-agent MCP server scoping - The applyTo: glob in .github/instructions/<NAME>.instructions.md (auto-loaded only when matching files are touched) - Body length cap (30,000 chars)

Everything else is **documentation discipline** — the agent follows its instructions because they're in the system prompt: - Handoff schema fields being present - Refusal of vague delegations - Evidence rule being followed - failure_condition observation rule - Hop limits ("3rd same-specialist delegation → escalate") - "Read the right instruction file before editing" - Definition of Done completeness

Don't conflate the two layers. When you say "the orchestrator can only delegate to project specialists," that's documentation in Copilot (no runtime allowlist for cross-agent invocation in current versions). When you say "the legal-compliance agent cannot edit files," that IS runtime-enforced via the tools: field.

6. Three-tier documentation

Project documentation should split into three clearly-labeled tiers:

Tier	Location	Purpose	Read by
Orientation maps	docs/ai-context/<topic>.md	50-150 lines per area; gotchas; cross-links to canonical	Agents (per task routing)
Canonical references	docs/<UPPERCASE>.md	Architecture, API, schema, business — full detail	Humans + agents (when deeper)
Frozen archive	docs/_archive/<date>/	Sprint reports, post-mortems, migration logs, snapshots	Audits only — never linked from active docs

This separation makes drift impossible. Active docs live in the first two tiers. The archive is append-only and never referenced by agents/instructions/active docs.

The orientation maps live in docs/ai-context/ even in a Copilot-only project — they're tool-agnostic, and using a tool-named directory (like docs/copilot-context/) would tie you to a single AI assistant.

7. Default Copilot stays default for inline completions

Do not put the orchestrator's persona in .github/copilot-instructions.md. The repository-wide instructions file is auto-loaded into EVERY Copilot interaction — including inline completions and code review. Loading the orchestrator's full persona there would: - Slow down inline completions - Pollute code-review prompts with delegation instructions - Force every interaction through the orchestrator's "delegate everything" body

.github/copilot-instructions.md should be a **thin router** — golden rules + workflow + cross-links. The orchestrator agent (.github/agents/<project>-orchestrator.md) is invoked explicitly when the user picks it from the agent dropdown or types @<project>-orchestrator in chat.

Three invocation modes coexist: - **Default** — inline completions + plain Chat (uses repository-wide instructions only) - **Specialist mode** — Chat with a specific specialist agent selected - **Orchestrator mode** — Chat with @<project>-orchestrator for cross-domain work

See docs/06-INVOCATION-MODES.md.

What these principles get you

A team can work in different IDEs (VS Code / JetBrains / Visual Studio) and on github.com (cloud agent, code review) with consistent behavior. Specialists never inherit baggage from unrelated work. Orchestrator decisions are auditable (every claim cites evidence). Hallucinations don't compound across hops. New engineers see a clean .github/ layout that tells them where to look.

The cost is upfront discipline: you spend a day writing the agent contracts, instruction files, and orientation maps. The payback is permanent — every future task benefits.

ewpage

02 — Architecture

The physical layout of files in a project that uses this framework.

The full picture

```
your-project/
│
├── .github/
│   ├── copilot-instructions.md          ← repo-wide router (auto-loaded everywhere)
│   ├── agents/                          ← orchestrator + specialists
│   │   ├── <project>-orchestrator.md
│   │   ├── <specialist-1>.md
│   │   ├── <specialist-2>.md
│   │   └── ...
│   ├── instructions/                    ← path-globbed invariants
│   │   ├── <domain-1>.instructions.md   (with applyTo: "src/api/**/*.ts")
│   │   ├── <domain-2>.instructions.md
│   │   └── ...
│   ├── prompts/                         ← repeatable workflows (slash commands)
│   │   ├── <workflow-1>.prompt.md       (invoked as /<workflow-1>)
│   │   ├── <workflow-2>.prompt.md
│   │   └── ...
│   └── chatmodes/                       ← optional: persona-based chat modes
│       └── <persona>.chatmode.md
│
├── AGENTS.md                            ← optional cross-AI metadata file
│                                          (also recognized by other AI tools)
│
├── docs/
│   ├── ai-context/                      ← orientation maps (read by agents)
│   │   ├── INDEX.md
│   │   ├── HANDOFF_SCHEMA.md
│   │   ├── ORCHESTRATION_SPOONFEEDER.md
│   │   └── <area>-experience.md         ← per-domain orientation, 50-150 lines each
│   ├── ARCHITECTURE.md                  ← canonical refs (full detail)
│   ├── API.md
│   ├── <YOUR-OTHER-CANONICAL-DOCS>.md
│   └── _archive/                        ← frozen historical
│       ├── README.md
│       └── <YYYY-MM>/
│
└── <your application code>/
```

What lives where, and why

.github/copilot-instructions.md (root router)

Auto-loaded by Copilot for EVERY interaction in this repository — including inline completions, Chat, code review, and cloud agent. Should be **under 200 lines**.

Contains: - Identity (one sentence about the project) - Golden rules (security, deployment, branching) - Mandatory workflow (numbered steps for every non-trivial task) - Routing guidance ("for X tasks, use the @ agent") - Cross-links to the spoonfeeder, INDEX, and canonical docs

Does NOT contain: - Long technical detail (lives in canonical docs) - Per-domain gotchas (lives in .github/instructions/) - The orchestrator's full persona (lives in .github/agents/<project>-orchestrator.md) - Sprint history or audit content (lives in docs/_archive/)

⚠ **Code review reads only the first 4,000 characters of this file.** If you need the router to be longer for Chat purposes, put critical-for-code-review content in the first 4,000 chars.

.github/agents/

One markdown file per agent. Frontmatter declares fields per the Copilot custom agents spec:

```yaml
---
name: <agent-name>                  # optional, defaults to filename
description: <required string>      # how the agent decides when to delegate
tools: ['tool-a', 'tool-b']         # optional allowlist
mcp-servers: { ... }                # optional per-agent MCP scoping
model: <model-name>                 # optional, IDE-specific
target: vscode | github-copilot     # optional environment scoping
disable-model-invocation: true|false  # optional, prevents auto-routing
user-invocable: true|false          # optional, controls manual selection
metadata: { key: value }            # optional annotations
---
```

Body is the agent's system prompt — up to 30,000 characters.

Always includes exactly one orchestrator (<project-slug>-orchestrator). Specialists are

added based on the project's natural domain boundaries (typically 5-12 specialists for a medium-complexity codebase).

See 03-AGENTS-GUIDE.md for how to design agents.

.github/instructions/

Path-globbed invariants — Copilot's direct equivalent of "rules" in other frameworks. Each file:

```
---
applyTo: "src/api/**/*.ts,src/server/**/*.ts"
excludeAgent: "code-review"         # optional: don't apply during code review
---

# API Layer Rules

## Hard rules

### <Rule title>
**Why:** <reason>
**How to apply:** <how>
```

Frontmatter applyTo: accepts glob patterns (comma-separated for multiples). When Copilot is editing a matching file, this instruction file is auto-loaded into the relevant prompts.

These are the **hard-won gotchas** of your codebase — things that broke production once and must not break again.

.github/prompts/

Reusable workflows invoked via slash command. Each file:

```
---
description: <one sentence — appears in /command picker>
---

<the prompt body — instructions, may include ${input:variable:hint} placeholders>
```

In Copilot Chat, type /<filename-without-extension> to invoke. Example: .github/prompts/investigate-bug.prompt.md is invoked as /investigate-bug.

Available in: VS Code, Visual Studio, JetBrains. Currently in **public preview**.

Use prompt files for repeatable multi-step workflows like bug investigation, feature build, QA pass, compliance review. They're the Copilot equivalent of "skills."

.github/chatmodes/ (optional)

Older format predating custom agents. Each file:

```
---
description: <one-sentence persona description>
tools: ['tool-a', 'tool-b']
model: <optional>
---

<persona system prompt>
```

Selected from the Chat dropdown.

Recommendation: prefer .github/agents/ over .github/chatmodes/ for new projects. .agent.md is the newer/preferred format. Keep chat modes only if you have legacy chat-mode files you don't want to migrate.

AGENTS.md (optional, recommended for cross-AI projects)

A cross-AI standard file recognized by GitHub Copilot, Claude, Gemini, and other AI assistants. If your project uses multiple AI tools, put shared cross-AI metadata here.

Format: plain markdown describing the project, agents, and conventions.

docs/ai-context/

Orientation maps. Per-area guides Copilot reads at the start of any task in that area. Cheaper than reading thousand-line canonical docs.

Format: 50-150 lines per file.

Audience: AI agents (primary), humans (secondary).

Naming: <area>-experience.md or <area>-<concern>.md.

Special files: - INDEX.md — task-type → docs + agent map - HANDOFF_SCHEMA.md — bidirectional handoff schema - ORCHESTRATION_SPOONFEEDER.md — human-facing usage guide

The docs/ai-context/ location is tool-agnostic so the same orientation maps work if you later add Claude Code, Gemini Code Assist, or other AI tools.

docs/<UPPERCASE>.md

Canonical references. Full detail. Updated when the underlying truth changes. As long as needed (no artificial line cap).

Common files: ARCHITECTURE.md, API.md, TECH_STACK.md, CHANGELOG.md, PRODUCT_REQUIREMENTS.md, BUSINESS_DOCUMENT.md.

docs/_archive/

Frozen historical material. Date-prefixed subdirectories (<YYYY-MM>/). Active docs never link here.

Required: _archive/README.md documenting the archive convention.

Why this layout works

Predictability for Copilot

Every agent knows where to find: - Its own contract: .github/agents/<self>.md - Repository-wide rules: .github/copilot-instructions.md (auto-loaded) - Path-specific rules: .github/instructions/*.instructions.md (auto-loaded when editing matching files) - Reusable workflows: .github/prompts/*.prompt.md (invoked via /<name>) - The handoff schema: docs/ai-context/HANDOFF_SCHEMA.md - Orientation per topic: docs/ai-context/<area>.md - Canonical references: docs/<UPPERCASE>.md

Predictability for humans

A new engineer joining the project sees: - .github/copilot-instructions.md — tells them how Copilot is configured - .github/agents/, .github/instructions/, .github/prompts/ — Copilot config files - docs/ — documentation - Application code in its standard tech-stack location

They can ignore .github/ agents/instructions/prompts if they don't use Copilot.

Minimal coupling to tech stack

Nothing in .github/ or docs/ai-context/ cares about your framework, language, or deploy target. The agents reference paths in your actual codebase, but the agent FILES themselves are pure markdown. You can reorganize your codebase entirely and only need to update the applyTo: globs and the orientation maps' cross-links.

What does NOT belong in this framework

- **Application code.** Stays where your tech stack puts it.
- **Build config.** Stays at root.
- **CI workflows.** Stays in .github/workflows/.
- **Test files.** Stays in your tests directory.
- **Generated artifacts.** Gitignored.

The framework is **purely additive** to whatever else lives in your repo.

Variations

Monorepo

For monorepos, you can either: - Have one .github/ at the root that knows about all packages, or - Have one AGENTS.md per package + a root .github/copilot-instructions.md that orchestrates across packages

The first is simpler. The second is more isolated. Pick based on whether tasks routinely cross package boundaries.

Mobile + web split

If your project has separate mobile and web codebases in one repo, your

orchestrator typically delegates to a mobile-ui specialist OR a web-ui specialist. They can share a backend-api specialist for the API-side work.

Multi-tenancy / multi-product

If you serve multiple products from one codebase, consider per-product orientation maps and per-product instructions. The orchestrator can be one or per-product depending on whether tasks routinely cross products.

Heavy infra / DevOps focus

Add specialists for infrastructure, observability, release-automation. Their instruction files cover Terraform/CDK/Kubernetes/Helm conventions.

ML / data heavy

Add specialists for data-pipeline, model-training, inference-serving. Their orientation maps cover dataset locations, experiment tracking, model versioning.

The architecture flexes; the principles don't.

Organization-level customization (Business / Enterprise)

If you have GitHub Copilot Business or Enterprise, you can also configure:

- **Organization custom instructions** (UI at organization settings) — applies to Chat on github.com, code review, and cloud agent across all org members
- **Organization-level custom agents** — /agents/<NAME>.md in a .github-private repository at the org or enterprise level

Use organization-level configuration for company-wide policies (e.g. "always cite the file:line for security-sensitive code"). Use repository-level for project-specific specialists and rules.

Precedence (highest to lowest): Personal > Repository > Organization. All sets are provided to Copilot when relevant.

ewpage

03 — Agents Guide (GitHub Copilot)

How to design the orchestrator and specialists for your project using GitHub Copilot's custom agents feature.

The orchestrator

Every project has exactly **one** orchestrator. Naming convention: <project-name>-orchestrator (e.g. acme-orchestrator, payments-platform-orchestrator).

Responsibilities

- Read incoming task. Restate it.
- Identify affected roles / domains.
- Read minimum docs (docs/ai-context/INDEX.md → 1-3 orientation maps).
- Pick the right specialists (typically 1-3 per task).
- Issue structured handoffs with claims + evidence + failure_condition.
- Receive returns. Verify rejected_claims. Re-verify unverified_claims before propagating.
- Aggregate findings into a final summary with the project's "Definition of Done."

Constraints

- **Restricted tool set** — primarily Read/Search tools, NOT edit tools. The orchestrator coordinates; specialists implement.
- **disable-model-invocation: false** — auto-routable when the task is complex.
- **user-invocable: true** — explicitly selectable from agent dropdown.

Frontmatter template

```
---
name: <project>-orchestrator
description: Main task dispatcher for <Project>. Use for any medium/complex task. Picks the minir
tools: ['codebase', 'search', 'usages', 'problems', 'changes']
target: github-copilot
user-invocable: true
disable-model-invocation: false
---
```

(Field reference: see Custom agents configuration — exact tool names depend on Copilot extension version.)

Specialists

A specialist owns one domain. Specialists are domain-shaped, not technology-shaped.

Bad: react-specialist, typescript-specialist. Good: frontend-ui, payments, realtime-sync, compliance.

How many do you need?

Project size	Specialist count
Solo project / prototype	0-2 (just use plain Copilot Chat)
Small team / single product	4-6
Medium codebase / multiple roles	6-10
Large codebase / multi-product	10-15

More than 15 is a smell — you're probably making them too narrow.

Common specialists by project type

Web app (any stack)

Specialist	Domain
frontend-ui	UI components, routing, state management, client-side interactions
backend-api	API routes, server logic, request handling
database	Schema, migrations, query layer, indexes
auth-security	Authentication, authorization, sessions, secrets
qa-functional	Browser/E2E testing, regression checks
release-devops	Build, deploy, env config, observability

Mobile app

Specialist	Domain
mobile-ui	Screens, navigation, gestures, platform-specific UX
mobile-state	State management, persistence, offline sync
mobile-native	Platform bridges (iOS/Android native or Flutter plugins)
backend-api	Server-side endpoints
qa-mobile	Device testing, simulator/emulator flows

Backend / API service

Specialist	Domain
api-routes	HTTP/gRPC route handlers
domain-logic	Core business logic
data-layer	DB access, repositories, ORM patterns
integrations	Third-party API clients
observability	Logging, metrics, tracing
release-devops	Deploy, env, secrets

Cross-cutting (add to most projects)

Specialist	Domain
product-flow	Acceptance criteria, cross-role behavior (LIGHT EDITS ONLY)
qa-functional	Tests across the system
security-privacy	Auth/sensitive-data review (REVIEW-ONLY)
legal-compliance	Regulatory review (REVIEW-ONLY)
release-devops	Build/deploy/cron/env
context-librarian	Maintains .github/, docs/ai-context/, archives drift

REVIEW-ONLY specialists

Some specialists should explicitly **not** edit code. Tag them in the description, give them only read tools (no edit_files / create_file / apply_patch), and document the constraint in their I CANNOT section.

Examples: - legal-compliance — flags risks; never claims final legal advice - security-privacy — identifies vulnerabilities; doesn't ship fixes - architecture-review — reviews proposed designs; doesn't implement

Frontmatter for REVIEW-ONLY:

```
---
name: legal-compliance
description: REVIEW-ONLY. Flags <list of regulations> risks. Produces attorney-checklist content.
tools: ['codebase', 'search', 'usages', 'problems', 'changes', 'fetch']
target: github-copilot
user-invocable: true
disable-model-invocation: false
---
```

Notably absent: edit-file tools, terminal/run tools. The harness physically prevents this agent from modifying files.

⚠ **Verify your IDE's exact tool names.** The tools: array names are tool identifiers Copilot recognizes — they vary slightly between extension versions and IDEs. The list above (codebase, search, usages, problems, changes, fetch) is a common read-only set. Check your IDE's agent configuration UI to see the current valid identifiers, or consult the Copilot documentation.

Implementation specialists

Get edit/write/terminal access plus relevant MCP server scope.

Frontmatter for an implementation specialist:

```
---
name: backend-api
description: Builds and fixes API routes, server logic, request handling. Coordinates with database
tools: ['codebase', 'search', 'usages', 'problems', 'changes', 'edit_files', 'apply_patch', 'runCommand
target: github-copilot
user-invocable: true
disable-model-invocation: false
---
```

For specialists that need MCP server access (e.g. browser testing, database introspection), declare via mcp-servers::

```
---
name: qa-functional
description: Browser-based E2E testing and regression checks across roles.
tools: ['codebase', 'search', 'usages', 'changes', 'edit_files', 'apply_patch', 'runCommands']
mcp-servers:
  playwright:
    type: 'local'
    command: 'npx'
    args: ['-y', '@modelcontextprotocol/server-playwright']
    tools: ['*']
---
```

(Exact MCP server config syntax may vary — check the Copilot MCP documentation.)

Agent body structure (every agent)

Every agent file body should include these sections in order:

```
# <Agent Display Name>

<2-sentence description of the agent's domain.>

## When to use

- Bullet list of task types this agent owns

## When NOT to use

- Bullet list of task types that belong to other agents

## Required reading

1. `docs/ai-context/INDEX.md` — task → docs map
2. <area-specific orientation map>
3. **`.github/instructions/<your-instructions>.instructions.md`** is auto-loaded when editing matchi
4. <other rules / canonical refs>

## Incoming handoff validation

<paste the standard incoming handoff validation block — see template>

## Return schema (required)

<paste the standard return schema block — see template>

## I CAN

- Bullet list of what this agent is allowed to do

## I CANNOT

- Bullet list of what this agent must refuse
- Always include: "Push to main" (or your equivalent protected branch)

## Definition of Done

1. Files changed (paths)
2. Instruction files referenced before editing
3. Tests run
4. Risks remaining
5. (any agent-specific Definition of Done items)

## Cross-links

- `<related instruction file>`
- `<related orientation map>`
- `<canonical doc>`
```

The “Incoming handoff validation” and “Return schema” sections are **identical across all specialists** — they enforce the universal handoff contract. See templates/specialist-agent.md.template.

Cross-agent invocation

In Copilot, one agent can reference another via the agent tool alias (when configured):

@<other-specialist-name> <task-with-handoff-yaml-block>

This is the equivalent of Claude Code’s Agent(specialist-1, specialist-2) allowlist — but Copilot doesn’t expose a per-agent allowlist for which other agents can be invoked. The orchestrator’s body documents which specialists exist; respecting the routing is documentation discipline.

Naming conventions

- Lowercase with hyphens: backend-api, frontend-ui, legal-compliance
- Filename matches name: backend-api → backend-api.md
- Be specific but not over-narrow: payments good, stripe-webhook-handler too narrow
- Cross-cutting concerns get their own agent: qa-functional, security-privacy, not folded into another specialist

When to add a new specialist vs. extend an existing one

Add a new specialist when: - The domain has its own gotchas (would warrant its own instruction file) - Tasks in that domain frequently arrive - The existing specialists’ “I CANNOT” sections would all reject this work

Extend an existing specialist when: - The work overlaps with an existing specialist's domain - The new domain is small (fits in 1-2 paragraphs of the existing agent's body) - You haven't seen 3+ tasks in this domain yet

Anti-patterns

Technology-named specialists. typescript-specialist doesn't have a clear domain.

Layer-named specialists. database-specialist is fine if "database" means a domain (schema, migrations); not fine if it means "anyone touching SQL anywhere should consult me."

Specialists with overlapping tools and overlapping descriptions. Confuses the auto-routing logic.

Specialists with no tools: field. They inherit ALL tools, defeating defense-in-depth. Always declare tools explicitly.

Specialists whose "I CAN" includes "approve production deploys." That's the project owner's call.

Putting the orchestrator's persona in .github/copilot-instructions.md. That file is loaded for inline completions and code review too — pollutes everything. Keep the orchestrator persona in .github/agents/<project>-orchestrator.md, separate.

A note on cloud agent vs IDE chat agents

The same .github/agents/<NAME>.md file works for both: - **IDE Chat custom agents** (VS Code, JetBrains, etc.) - **Cloud agent on github.com** (autonomous agent that takes issue/PR assignments)

Some properties may behave differently between environments: - The cloud agent runs in true context isolation - The IDE chat agent runs within the active Chat session's context - The target: field can scope an agent to one or the other (vscode vs github-copilot) - Some IDE-specific properties (e.g. argument-hint, handoffs in VS Code) are not supported in the cloud agent

For most projects, write agents that work in BOTH environments by avoiding IDE-specific properties. Use target: only when an agent genuinely needs to be exclusive to one environment.

ewpage

04 — Handoff Schema (GitHub Copilot)

The bidirectional schema for orchestrator ↔ specialist communication. This is the framework's central artifact.

The schema is **identical** to the equivalent in the Claude Code framework — it's tool-agnostic. What changes is how it's invoked: in Copilot, the orchestrator emits the YAML block when it @-mentions a specialist or when the cloud agent is delegating work.

Why a schema

Without structure, handoffs are free-form prompts. Specialists guess what's expected. Orchestrators don't commit to specific claims. Cascading hallucinations compound across hops because nothing forces evidence-to-claim binding at the boundary.

The schema closes both gaps with two YAML blocks (one per direction) and one universal evidence rule.

Universal evidence rule

> **No evidence, no claim.** If a claim cannot be tied to a file, line, table, column, rule, doc, test, or command output, it must be marked as an assumption (confidence: low outbound, or moved to unverified_claims inbound) and cannot be passed downstream as fact.

This rule applies to both directions of every handoff. It is repeated verbatim in the orchestrator's outbound-discipline section and in every specialist's incoming-handoff-validation section.

Outbound: orchestrator → specialist

When the orchestrator delegates to a specialist (via `@<specialist-name>` in Chat or by spawning the specialist in the cloud agent flow), the prompt body MUST include this YAML block at the top:

```yaml
handoff:
  schema_version: 1
  handoff_id: <short unique id, e.g. h-2026-05-02-a3f>
  from_agent: <orchestrator-name>
  to_agent: <specialist-name>
  goal: <one sentence — what done looks like>
  task_class: <bug | feature | review | qa | refactor | audit>
  affected_roles: [<role1>, <role2>, ...]
  claims:
    - claim: <factual statement the orchestrator commits to>
      evidence: <path/to/file.ts:42 OR instruction:.github/instructions/foo.instructions.md OR doc:doc
      confidence: <high | medium | low>
  verify_before_acting:
    - <claim or assumption the specialist MUST re-verify against source before editing>
  failure_condition: <one sentence — observable evidence that proves the orchestrator's hypothes
  in_scope:
    - <path or component the specialist may edit>
  out_of_scope:
    - <path or component the specialist must NOT touch — even if "while we're in there">
  instructions_to_read:
    - <.github/instructions/<NAME>.instructions.md path that applies to in-scope files, OR "specialis
  expected_artifacts:
    - <file changed, test added, summary section, etc.>
  hop_context:
    parent_task: <one line — why the orchestrator is delegating>
    previous_specialist: <agent name or "none">
    previous_findings_summary: <one line — what came back, or "n/a">
```

Field requirements (outbound)

Field	Required	Notes
schema_version	yes	Currently `1`. Bump on breaking changes.
handoff_id	yes	Short unique id. Specialist echoes it on return so they pair.
from_agent	yes	Always the orchestrator's name.
to_agent	yes	The specialist's `name` field from its `.github/agents/<NAME>.md` frontmatter.
goal	yes	One sentence with measurable outcome. "Fix" or "improve" without an outcome is too vague.
task_class	yes	One of the six values.
affected_roles	yes	Non-empty list.
claims	yes (may be empty list)	Every entry MUST have `evidence`. Use `confidence: low` for assumptions.
verify_before_acting	yes if task touches code	Empty allowed only for pure-research handoffs.
failure_condition	yes	One sentence. The inverse of `verify_before_acting`.
in_scope	yes	Non-empty list.
out_of_scope	yes	Empty list `[]` is valid. Missing key is not.
instructions_to_read	yes	Use `["specialist will discover from applyTo globs"]` if you don't know which instructions apply.
expected_artifacts	yes	Drives the specialist's `files_changed` / `tests_run` later.
hop_context	yes	All three sub-fields required. Use `none` / `n/a` for first hop.

Inbound: specialist → orchestrator (return schema)

Every specialist response MUST end with this YAML block:

```yaml
return:
  schema_version: 1
  handoff_id: <copy from inbound — pairs the response>
  from_agent: <specialist-name>
  to_agent: <orchestrator-name>
  status: <completed | blocked | needs_clarification | claim_rejected>
  verified_claims:
    - claim: <orchestrator claim that was confirmed against source>
      evidence: <path:line or command output the specialist saw>
  rejected_claims:
    - claim: <orchestrator claim that turned out to be wrong>
      evidence: <what the specialist actually found>
  unverified_claims:
    - claim: <orchestrator claim that was not checked>
      reason: <why — out of scope, infra not available, etc.>
  evidence_used:
    - <path:line, instruction file, doc anchor, or command output>
  files_changed:
    - <path:line range and one-line summary>
  tests_run:
    - <command + result>
  risks:
    - <risk + mitigation or "none">
  recommended_next_agent:
    agent: <specialist-name or "none">
    why: <one line>
  do_not_pass_downstream_without_verification:
    - <claim or finding the orchestrator must NOT propagate as fact>
```

Field requirements (inbound)

Field	Required	Notes
schema_version	yes	Must match outbound.
handoff_id	yes	Must echo inbound id verbatim.
from_agent / to_agent	yes	Self-explanatory.
status	yes	completed only if all verify_before_acting items checked. Use claim_rejected when an orchestrator claim was wrong.
verified_claims	yes (may be [])	Empty list is valid. Missing key is not.
rejected_claims	yes (may be [])	Empty list is valid. Non-empty implies status: claim_rejected or blocked.
unverified_claims	yes (may be [])	Anything you accepted without re-checking goes here.
evidence_used	yes	Real paths/commands. “I read the codebase” is not evidence.
files_changed	yes (may be [])	Empty list valid for review/audit handoffs.
tests_run	yes (may be [])	Empty list valid for pure-doc changes.
risks	yes	Use ["none"] if there are genuinely none.
recommended_next_agent	yes	agent: "none" if work is complete.
do_not_pass_downstream_without_verification	yes (may be [])	Cascading-hallucination defense.

Refusal

If the inbound handoff fails the field requirements, the specialist responds with ONLY this block, does no other work, and waits for re-issue:

```yaml
return:
  schema_version: 1
  handoff_id: <copy from inbound, or "missing" if absent>
  from_agent: <specialist-name>
  to_agent: <orchestrator-name>
  status: needs_clarification
  reason: handoff_schema_invalid
  missing_or_invalid_fields:
    - field: <field name>
      problem: <what's wrong — missing, vague, no evidence, etc.>
  required_action:
    - <what the orchestrator must add or fix before re-issuing>
```

Failure-condition observation rule

If at any point during the work the specialist observes the inbound `failure_condition`, it STOPS immediately and responds with the return schema using `status: claim_rejected` (or `needs_clarification` if ambiguous). It cites the triggering evidence in `evidence_used`. It does NOT push through a falsified premise.

Worked examples

Example 1 — Backend bug investigation

Outbound (orchestrator → backend-api):

```yaml
handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-bug-001
  from_agent: acme-orchestrator
  to_agent: backend-api
  goal: Find why /api/orders returns 500 for orders with > 100 line items.
  task_class: bug
  affected_roles: [customer]
  claims:
    - claim: The 500 originates in the order serialization step.
      evidence: src/api/orders/handler.ts:88
      confidence: high
    - claim: Order serialization runs synchronously and may exceed function timeout.
      evidence: instruction:.github/instructions/api.instructions.md
      confidence: medium
  verify_before_acting:
    - That src/api/orders/handler.ts:88 is still the serialization line.
    - That the timeout is actually exceeded (logs / metric, not just suspected).
  failure_condition: If logs show the 500 happens in the database query rather than serialization, S
  in_scope:
    - src/api/orders/handler.ts
    - src/api/orders/serializer.ts
  out_of_scope:
    - The order schema (database concern; separate specialist if migration needed).
  instructions_to_read:
    - .github/instructions/api.instructions.md
    - .github/instructions/observability.instructions.md
  expected_artifacts:
    - Root cause statement (file:line)
    - Smallest fix or "no bug — load test issue" verdict
  hop_context:
    parent_task: Customer reports cart > 100 items always errors at checkout.
    previous_specialist: none
    previous_findings_summary: n/a
```

Return (backend-api → orchestrator):

```yaml
return:
  schema_version: 1
  handoff_id: h-2026-05-02-bug-001
  from_agent: backend-api
  to_agent: acme-orchestrator
  status: claim_rejected
  verified_claims:
    - claim: src/api/orders/handler.ts:88 is the serialization line.
      evidence: src/api/orders/handler.ts:88 (confirmed via codebase search + read)
  rejected_claims:
    - claim: The 500 originates in serialization.
      evidence: Stack trace from production logs shows the failure is in the database query at src/db
  unverified_claims: []
  evidence_used:
    - src/api/orders/handler.ts:80-95
    - src/db/repositories/orderRepo.ts:140-150
    - usages search for "loadOrderLineItems" in src/
    - production logs (sampled 3 failing requests)
  files_changed: []
  tests_run: []
  risks:
    - Real fix requires pagination of the query. Out of this handoff's scope.
  recommended_next_agent:
    agent: database
    why: Pagination of orderRepo.loadOrderLineItems is the correct fix.
  do_not_pass_downstream_without_verification:
    - That the timeout value is actually 30s (read from env or config; I inferred from observed behav
```

Example 2 — Mobile feature build

Outbound (orchestrator → mobile-ui):

```yaml
handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-feat-002
  from_agent: acme-orchestrator
  to_agent: mobile-ui
  goal: Add a pull-to-refresh gesture on the OrderHistoryScreen.
  task_class: feature
  affected_roles: [customer]
  claims:
    - claim: OrderHistoryScreen lives at src/screens/OrderHistoryScreen.tsx.
      evidence: src/screens/OrderHistoryScreen.tsx
      confidence: high
    - claim: The app uses React Native's RefreshControl for pull-to-refresh elsewhere.
      evidence: instruction:.github/instructions/mobile-ui.instructions.md
      confidence: high
  verify_before_acting:
    - That OrderHistoryScreen uses a ScrollView or FlatList that can host RefreshControl.
  failure_condition: If OrderHistoryScreen renders a non-scrollable view (e.g. a Map or a custom vi
  in_scope:
    - src/screens/OrderHistoryScreen.tsx
    - src/screens/__tests__/OrderHistoryScreen.test.tsx (add test)
  out_of_scope:
    - The order data fetching logic (already exists; just call refetch).
  instructions_to_read:
    - .github/instructions/mobile-ui.instructions.md
  expected_artifacts:
    - Edit to OrderHistoryScreen.tsx adding RefreshControl
    - New test asserting onRefresh triggers refetch
  hop_context:
    parent_task: Product wants pull-to-refresh on order history per ACME-1234.
    previous_specialist: none
    previous_findings_summary: n/a
```

Example 3 — Compliance review (REVIEW-ONLY specialist)

Outbound (orchestrator → legal-compliance):

```yaml
handoff:
  schema_version: 1
  handoff_id: h-2026-05-02-rev-003
  from_agent: acme-orchestrator
  to_agent: legal-compliance
  goal: Review the new email opt-in flow for GDPR/CASL/CAN-SPAM compliance before launch.
  task_class: review
  affected_roles: [customer]
  claims:
    - claim: The opt-in flow records consent timestamp + IP + opt-in source.
      evidence: src/db/migrations/2026-05-01-add-consent-log.sql
      confidence: high
  verify_before_acting:
    - That the migration has been applied to staging.
    - That the consent log columns are actually written by the new opt-in handler.
  failure_condition: If the consent log table exists but the opt-in handler does NOT write to it, STOP
  in_scope:
    - REVIEW-ONLY. No code edits.
    - src/email/**
    - src/db/migrations/2026-05-01-*
  out_of_scope:
    - Any code changes (escalate to email-templates or backend-api specialists).
  instructions_to_read:
    - .github/instructions/security-privacy.instructions.md
  expected_artifacts:
    - Severity-ranked findings (blocker / high / medium / low)
    - Attorney-checklist of questions for counsel
  hop_context:
    parent_task: Pre-launch gate for new email opt-in feature.
    previous_specialist: none
    previous_findings_summary: n/a
```

Versioning

- schema_version: 1 is current.
- Additive changes (new optional fields) keep version 1.
- Breaking changes (renaming fields, changing field semantics) bump the version.
- Bumps require updating HANDOFF_SCHEMA.md, the orchestrator, and every specialist file in the same PR.

Why this schema is enough (without runtime hooks)

The schema is enforced by: 1. The orchestrator's instructions to ALWAYS emit the outbound block. 2. Every specialist's instructions to refuse delegations missing the block. 3. The orchestrator's instructions to consume the return block before merging work.

This is documentation enforcement — agents follow the rules because their instructions tell them to. It works because: - The Copilot harness at least enforces the tools: allowlist (so a misbehaving agent can't write where it shouldn't) - The orchestrator catches return-block violations (missing fields, wrong types) - Refusal is a one-shot rejection: if the specialist returns the refusal template, the orchestrator must re-issue with the missing fields

If GitHub Copilot eventually exposes pre-tool-use hooks (similar to other AI tools), you could add hard runtime enforcement — but the documentation enforcement covers the majority of value at minimal complexity.

ewpage

05 — Instructions and Prompts

Two patterns for capturing knowledge that doesn't belong in agents: **instruction files** (path-globbed invariants) and **prompt files** (repeatable workflows invoked via slash command).

Instruction files

Path-globbed invariants. Each file in .github/instructions/ has YAML frontmatter declaring which file paths it applies to. When Copilot is editing a matching file, the instruction file is auto-loaded into the prompt.

This is the **direct equivalent of "rules"** in other frameworks — and arguably

cleaner because the path globbing is in YAML frontmatter rather than in body text.

When to write an instruction file

Write one when: - A bug shipped to production once and must not recur - A pattern is non-obvious from reading the code (e.g. “this column looks like a string but it’s a JSON-encoded string in legacy rows”) - A constraint crosses multiple files but isn’t enforced by types/tests - An anti-pattern is tempting but wrong

Don’t write one for: - Things obvious from the code - Style preferences (lint config does that) - One-off decisions (PR descriptions do that) - Anything covered by an existing instruction file (extend the existing one)

Instruction file structure

```
---
applyTo: "src/api/**/*.ts"
excludeAgent: "code-review"
---

# <Domain Name> Instructions

## Hard rules

### <Rule title — short imperative>

**Why:** <one-sentence reason — usually a past incident or invariant>

**How to apply:** <2-4 sentences — what to do, what helper to use, what to check>

### <Next rule>

...

## What NOT to do

- Bullet list of anti-patterns

## Cross-links

- `<related orientation map>`
- `<related canonical doc>`
- `<related instruction file>`
```

Frontmatter fields

Field	Required	Notes
applyTo	yes	Glob pattern. Multiple patterns separated by commas: "**/*.ts,**/*.tsx".
excludeAgent	no	Prevent specific Copilot features from using this. Values: "code-review", "cloud-agent".

Glob patterns

Standard shell globs. Examples:

```
applyTo: "src/api/**/*.ts"
applyTo: "src/db/migrations/**/*.sql"
applyTo: "src/{lib,utils}/auth-*.ts"
applyTo: "components/payment/**/*.{tsx,jsx}"
applyTo: "ios/Runner/**/*.swift"
applyTo: "lib/screens/**/*.dart"
applyTo: "internal/handlers/**/*.go"
applyTo: "app/models/*.rb"
```

Globs are matched against the relative path from project root.

Limit on code review

⚠ **Code review reads only the first 4,000 characters of each instruction file.** If you need code-review-relevant rules, put them at the top of the file.

To exclude an instruction from code review entirely:

```
applyTo: "src/api/**/*.ts"
excludeAgent: "code-review"
```

Examples by tech

Backend / API

```
---
applyTo: "src/api/**/*.ts,src/server/**/*.ts"
---

### Always validate request body before reading database

**Why:** Three production incidents traced to unvalidated body data flowing to SQL parameters.

**How to apply:** Use `validateRequest(schema)` middleware in `src/lib/validation.ts`. Never re
```

Database / migrations

```
---
applyTo: "**/migrations/**/*.sql,prisma/schema.prisma"
---

### Migration order is sacred — never reorder, never edit applied migrations

**Why:** Migrations apply in filename order. Editing an applied migration silently diverges staging f

**How to apply:** New migrations get the next sequential timestamp prefix. Bug fixes to applied mi
```

Frontend / UI

```
---
applyTo: "src/components/**/*.{tsx,jsx},src/app/**/*.{tsx,jsx}"
---

### Never trust component props for security checks — re-validate on the server

**Why:** A user can edit any prop via React DevTools.

**How to apply:** Server-side handlers re-validate authorization. Props are for rendering, not auth
```

Mobile

```
---
applyTo: "src/screens/**/*.tsx,src/components/**/*.tsx"
---

### iOS auto-zooms on input focus when font-size < 16px

**Why:** iOS Safari WebView feature; jarring UX.

**How to apply:** Set `font-size: 16px` minimum on all `<input>`, `<select>`, `<textarea>`. For v
```

Anti-patterns in instruction files

Long rationale paragraphs. Rules should be ≤ 4 sentences each. Move long stories to the canonical doc.

Aspirational rules. “We should always…” — if it’s not enforced, it’s not a rule.

Rules that contradict other instruction files. Have the context-librarian agent reconcile.

Files with no `applyTo`. Without the glob, Copilot has no signal to load it.

Putting too many rules in one file. Group by domain, not by “all rules in one file.” If a file passes ~150 lines or 3,000 chars, split it.

Prompt files

Repeatable workflows invoked via slash command. Each file in `.github/prompts/` has frontmatter + body.

When to write a prompt file

Write one when: - The same kind of task arrives 3+ times - The task has multiple steps with verification between them - The cost of skipping a step is high (e.g. forgot to run security review before launching a new auth flow) - A junior team member would benefit from a guided checklist

Don’t write one for: - One-off tasks - Tasks where the steps vary too much per instance - Things that are really just a single agent invocation

Prompt file structure

```
---
description: <one-sentence description of when to use this — appears in /command picker>
---

<the prompt body — instructions, may include ${input:variable:hint} placeholders>

## Steps

### 1. <Step name>

<2-4 sentences>

### 2. <Next step>

...

## Definition of Done

1. <Artifact>
2. <Verification>
```

Frontmatter fields (current public preview)

Field	Required	Notes
description	yes	One sentence shown in the /command picker.
agent	optional	Specific agent to associate with this prompt.

(More fields may be supported by your IDE — check the Copilot prompt files documentation.)

Input placeholders

Inside the body, use ${input:NAME:HINT} for prompted inputs:

```
Explain the following code in a clear, beginner-friendly way:

Code to explain: ${input:code:Paste your code here}
Target audience: ${input:audience:Who is this explanation for?}
```

When invoked via /<filename>, Copilot prompts the user for each input.

Invocation

In Copilot Chat, type / to see the picker. Pick from the list, or type the prompt name: /<filename-without-extension>. Example: .github/prompts/investigate-bug.prompt.md → /investigate-bug.

Available in: VS Code, Visual Studio, JetBrains. Currently in **public preview** — features may evolve.

Common prompt files (most projects benefit from these)

.github/prompts/investigate-bug.prompt.md

Steps: restate bug → identify affected roles → read minimum docs → trace UI → state → API → DB → tests → match touched paths to instruction files → reproduce → classify severity → propose smallest fix → verify.

.github/prompts/build-feature.prompt.md

Steps: restate feature → identify roles → read minimum docs → run pre-feature checklist → write acceptance criteria per role → identify instruction-covered paths → plan smallest implementation → implement incrementally.

.github/prompts/qa-flow.prompt.md

Steps: identify flows in scope → read testing strategy + relevant orientation map → cover golden path + edge cases + cross-role → verify functional + data correctness → severity-rank findings.

.github/prompts/compliance-review.prompt.md

Steps: identify data involved → identify if regulated subjects involved → read compliance orientation map → apply standing checklist → flag risks per regulation

→ produce attorney checklist.

.github/prompts/context-refactor.prompt.md

For maintaining the framework itself: read current INDEX → categorize suspect content → move to correct home → consolidate duplicates → flag conflicts → keep .github/copilot-instructions.md small.

Instructions vs prompt files vs agents — when to use which

If the knowledge is…	Put it in…
A path-scoped invariant ("don't do X when editing Y")	Instruction file
A multi-step workflow that recurs (invoked manually)	Prompt file
A persona with its own scope and Definition of Done	Custom agent
A one-time decision rationale	PR description
Long-form architecture explanation	Canonical doc
Quick orientation for an area	Orientation map (docs/ai-context/<area>.md)

When something feels like it could be 2 of those: prefer instruction file > prompt file > agent (in that order). Instruction files are most precise (path-globbed, auto-loaded). Prompt files are next (named workflow, manual). Agents are heaviest (full persona).

Naming and lifecycle

Instruction files

- Lowercase with hyphens, .instructions.md suffix
- File name = domain name
- New gotcha from a new domain: create new file
- New gotcha from existing domain: append to existing file
- Stale instruction (the underlying constraint was fixed): delete it, don't leave it

Prompt files

- Lowercase with hyphens, .prompt.md suffix
- Filename becomes the slash command (investigate-bug.prompt.md → /investigate-bug)
- Used < 3x per quarter: consider deleting; the workflow probably wasn't recurrent enough

ewpage

06 — Invocation Modes

GitHub Copilot has more invocation modes than typical AI tools because it spans inline completions, Chat (in IDE and on github.com), Cloud Agent (autonomous), and CLI. Each surface has different rules for what gets loaded.

The five modes

Mode 1: Inline completions (autocomplete)

How: ghost text in the editor as you type.

Loads: - .github/copilot-instructions.md (auto) - .github/instructions/<NAME>.instructions.md matching the current file's path (auto)

Does NOT load: - Custom agents (.github/agents/) - Prompt files - Custom chat modes

Use for: typing assistance. Keep .github/copilot-instructions.md short for fast inline completions.

Mode 2: Chat in IDE (default)

How: Copilot Chat sidebar in VS Code / JetBrains / Visual Studio.

Loads: - .github/copilot-instructions.md (auto) - .github/instructions/<NAME>.instructions.md matching files in current chat context (auto)

Available on demand: - Custom agents (selectable from agent dropdown or @-mention) - Prompt files (invokable via /<name>) - Custom chat modes (selectable from mode dropdown)

Use for: most interactive work. Combine @<specialist> + /<workflow> for power.

Mode 3: Chat with a specific agent (specialist mode)

How: Select a specific agent from the agent dropdown, OR type @<agent-name> in chat.

Loads: - The selected agent's .github/agents/<name>.md body as system prompt - The agent's tools: allowlist is enforced - .github/copilot-instructions.md (still auto-loaded) - .github/instructions/ matching paths (still auto-loaded)

Use for: narrow domain work. Examples: - @frontend-ui for UI session - @backend-api for API work - @qa-functional for QA pass with browser MCP - @legal-compliance for review-only session

The agent's tools: allowlist physically restricts what it can do (e.g. legal-compliance literally cannot edit files because it has no edit tools).

Mode 4: Chat with the orchestrator (multi-domain mode)

How: Select your <project>-orchestrator agent OR type @<project>-orchestrator <task> in chat.

Loads: the orchestrator's body as system prompt + everything from Mode 3.

Use for: - Anything spanning more than one role - Anything spanning more than one feature area - Data integrity, payments, security/privacy, compliance work - Bug investigations where root cause may cross layers - Feature builds where acceptance criteria need writing before implementation - Pre-release QA across multiple roles - Anything where the scope is ambiguous

The orchestrator delegates to specialists by @<specialist> mentions in its responses (you may see it propose to invoke a specialist; confirm or paste the recommended next step).

Mode 5: Cloud Agent (autonomous)

How: Assign a Copilot agent to a GitHub issue or PR. The cloud agent runs autonomously on github.com infrastructure, makes changes, and opens a PR for review.

Loads: - .github/copilot-instructions.md (auto) - .github/instructions/<NAME>.instructions.md matching files it edits (auto) - The custom agent assigned (from .github/agents/<NAME>.md) - For org/enterprise: org-level custom agents from .github-private/agents/<NAME>.md

Strong context isolation: the cloud agent runs in a fresh environment per task. No leakage from previous sessions.

Use for: well-defined, self-contained tasks where you'd otherwise hand off to a junior engineer. Not for ambiguous work that needs back-and-forth.

Mode 6: Copilot CLI

How: gh copilot suggest / gh copilot explain in the terminal.

Loads: Copilot CLI has its own instruction system (separate from IDE). See Copilot CLI custom instructions.

Use for: terminal command suggestions and explanations.

Decision tree

```
Is this an editor/typing assist task?
  YES → Inline completions (no choice — that's what's running)
  NO ↓

Is this a quick question about code (read-only)?
  YES → Plain Chat (no agent selected)
  NO ↓

Is this narrow single-domain work (one specialist's territory)?
  YES → Chat with that specialist agent (@<specialist>)
  NO ↓

Is this cross-domain work, ambiguous scope, or production-sensitive?
  YES → Chat with orchestrator (@<project>-orchestrator)
  NO ↓

Is this a well-defined, self-contained task you'd assign to a junior?
  YES → Assign issue to Cloud Agent
  NO ↓

Is this a terminal-related question?
  YES → gh copilot suggest / explain
```

Why we don’t put the orchestrator persona in .github/copilot-instructions.md

The repository-wide instructions file is auto-loaded into: - Inline completions (every keystroke!) - Chat sessions (every prompt) - Code review (PR diff comments) - Cloud agent (every task)

Putting the orchestrator’s full delegation persona there would: - **Slow down inline completions** (every keystroke loads the persona) - **Pollute code review** with delegation instructions (“delegate this fix to backend-api”) - **Force every interaction** through the orchestrator’s “delegate, don’t implement” pattern

Instead, `.github/copilot-instructions.md` is a **thin router** — golden rules + workflow + cross-links. The orchestrator persona is in `.github/agents/<project>-orchestrator.md` and only loads when explicitly selected.

Mode comparison table

Mode	Loads orchestrator persona?	Specialists invokable?	Tool allowlist enforced?	Context isolation
Inline completions	No	No	N/A	n/a
Plain Chat	No	Yes (manual @)	Default Chat tools	Within Chat session
Specialist Chat	No	(just this one)	Yes (per agent’s `tools:`)	Within Chat session
Orchestrator Chat	Yes	Yes (via orchestrator’s @ calls)	Yes (per agent’s `tools:`)	Within Chat session
Cloud Agent	If assigned	If multi-agent	Yes (per agent’s `tools:`)	Strong (fresh env)
Copilot CLI	n/a (separate)	n/a	n/a	Separate context

Practical recipe

A well-functioning team uses several modes:

```
# Quick fix (90% of casual work)
Open Copilot Chat, no agent selected
"Update the button label from 'Sign In' to 'Sign in'"

# Specialist deep-dive (focused domain work)
Select @frontend-ui in Chat
"Audit our Suspense boundaries for client-component leaks"

# Full orchestrated work (multi-domain, production-sensitive)
Select @<project>-orchestrator in Chat
"Add a new payment method that requires KYC review and cron-driven activation"

# Autonomous bug fix (well-defined, junior-level)
Assign GitHub issue to Copilot
The cloud agent investigates, fixes, opens PR
```

Anti-patterns

Using plain Chat for everything. Cross-domain work flooded with context. Specialists never invoked.

Using @<orchestrator> for typos. Forced delegation overhead. Specialist spawn for a one-line fix.

Not knowing which mode you're in. Check the agent dropdown in Chat. If you're not sure, you're probably in the wrong mode.

Putting the orchestrator persona in .github/copilot-instructions.md. See section above — it pollutes everything.

Letting the Cloud Agent take ambiguous tasks. Cloud Agent works best on tasks with clear scope and acceptance criteria. Ambiguity → bad PRs → human cleanup.

When you change projects

Each project has its own .github/agents/, so @<project>-orchestrator is project-specific. The orchestrator name should be <project>-orchestrator (e.g. acme-orchestrator, not just orchestrator) so that when you have multiple repos open simultaneously, you can tell at a glance which orchestrator is active.

Surface support matrix (current)

Feature	VS Code	JetBrains	Visual Studio	github.com Chat	Clou Ager
.github/copilot-instructions.md					
.github/instructions/*.instructions.md (auto-load on path match)					
.github/prompts/*.prompt.md (slash commands)					
.github/agents/*.md (custom agents)					
.github/chatmodes/*.chatmode.md (chat modes)		(older)	(older)		
MCP servers per agent				partial	partia

(Surface support evolves rapidly — verify against current Copilot release notes.)

ewpage

07 — Folder Structure

How to organize Copilot config and documentation so the AI (and humans) always know where to look.

The Copilot config tier

```
.github/
├── copilot-instructions.md          ← repo-wide router (auto-loaded everywhere)
├── agents/                          ← orchestrator + specialists
│   ├── <project>-orchestrator.md
│   └── <specialist>.md
├── instructions/                    ← path-globbed invariants (frontmatter has applyTo:)
│   └── <domain>.instructions.md
├── prompts/                         ← repeatable workflows (slash commands)
│   └── <workflow>.prompt.md
└── chatmodes/                       ← optional: persona-based modes
    └── <persona>.chatmode.md
```

The documentation tier

```
docs/
├── ai-context/                      ← TIER 1: orientation maps (read by Copilot agents)
│   ├── INDEX.md
│   ├── HANDOFF_SCHEMA.md
│   ├── ORCHESTRATION_SPOONFEEDER.md
│   └── <area>-experience.md
│
├── ARCHITECTURE.md                  ← TIER 2: canonical references
├── API.md
├── TECH_STACK.md
├── <UPPERCASE>.md
│
└── _archive/                        ← TIER 3: frozen historical
    ├── README.md
    └── <YYYY-MM>/
```

Tier 1 — Orientation maps (docs/ai-context/)

Purpose: Per-area guides Copilot reads at the start of any task in that area.

Format: 50-150 lines per file.

Audience: Copilot agents (primary), humans (secondary).

Naming: <area>-experience.md or <area>-<concern>.md. Examples: - customer-experience.md - admin-experience.md - auth-and-sessions.md - payments.md - realtime-sync.md - data-pipeline.md

Body sections: 1. **2-sentence orientation.** What this area is, what it isn't. 2. **Key file paths.** 3. **3-5 gotchas worth knowing.** 4. **Cross-links** to canonical refs and to related instruction files.

Special files: - INDEX.md — task-type → docs + agent map - HANDOFF_SCHEMA.md — bidirectional handoff schema - ORCHESTRATION_SPOONFEEDER.md — human-facing usage guide - MIGRATION_LEDGER.md (optional) — tracks doc/structure migration progress - LEGACY_BACKUP.md (optional) — frozen pre-refactor snapshot

> **Why docs/ai-context/ and not docs/copilot-context/?** The orientation maps are tool-agnostic. If you later add Claude Code, Gemini Code Assist, or other AI tools, they'll all read from the same place. Don't tie this folder to a single AI vendor.

Tier 2 — Canonical references (docs/<UPPERCASE>.md)

Purpose: Authoritative source of truth. Full detail. Updated when underlying truth changes.

Format: As long as needed. No artificial line cap.

Audience: Humans (primary), agents (when deeper detail needed).

Naming: UPPERCASE.md. Convention signals "this is canonical."

Common files: ARCHITECTURE.md, API.md, TECH_STACK.md, CHANGELOG.md, PRODUCT_REQUIREMENTS.md, BUSINESS_DOCUMENT.md.

Cross-referencing: - Orientation maps in ai-context/ link DOWN to canonical docs. - Canonical docs do NOT link UP to orientation maps. - Agents link to either, depending on depth needed.

Tier 3 — Frozen archive (docs/_archive/)

Purpose: Historical material no longer authoritative but worth preserving for audits.

Format: Whatever the original was — markdown, HTML, PNG, CSV.

Audience: Audits only. Active docs never link here.

Naming: Date-prefixed subdirectories: <YYYY-MM>/<file>.

Required: _archive/README.md documenting: - What this directory is - Three rules: don't link from active docs, don't delete, don't move back to root - "Known link rot" section if you migrated from a structure with cross-links

What goes where — decision tree

You have a doc to write or move. Where?

Is it a per-area gotcha guide of 50-150 lines?
  → docs/ai-context/<area>.md

Is it the system architecture, API reference, or another full-detail doc?
  → docs/<UPPERCASE>.md

Is it a path-globbed invariant ("don't do X when editing Y")?
  → .github/instructions/<domain>.instructions.md (with applyTo: in frontmatter)

Is it a multi-step workflow that recurs (manually invoked)?
  → .github/prompts/<workflow>.prompt.md

Is it an agent persona definition?
  → .github/agents/<name>.md

Is it a sprint report, audit, post-mortem, or dated snapshot?
  → docs/_archive/<YYYY-MM>/<file>.md

Is it a one-time decision rationale?
  → PR description or commit message — NOT docs/

Is it just routing ("for X tasks, use Y agent")?
  → .github/copilot-instructions.md (kept SHORT) or docs/ai-context/INDEX.md

Folder-level conventions

Root level

The root contains ONLY: - Tech-stack config files (package.json, tsconfig.json, Cargo.toml, go.mod, etc.) - Build/deploy config (Dockerfile, docker-compose.yml, etc.) - CI config (.github/workflows/, .gitlab-ci.yml) - Test runner config (playwright.config.ts, pytest.ini, etc.) - The framework files (AGENTS.md if used, .github/, docs/) - Application directories (src/, app/, lib/, pkg/, etc.) - Static assets (public/, assets/)

The root does NOT contain: - Loose orphan scripts (move to scripts/_archive/ if not actively used) - Screenshots or images (move to docs/_archive/screenshots/ or assets/) - Backup env files (gitignore + delete from history if secrets were committed) - Generated artifacts (gitignore) - Stub directories with one file (consolidate elsewhere)

Tracked-cache cleanup

Common directories to gitignore (often tracked by accident): - IDE caches (.vscode/, .idea/ — at minimum gitignore the auto-generated parts) - Test runner per-run state (test-results/, coverage/) - Database CLI temp dirs - Build artifacts (.next/, dist/, build/, __pycache__/) - Local logs (logs/)

Env file safety

Add broad gitignore patterns:

```
.env
.env.*
!.env.example
.env*.bak*
.env*staging_tmp
.env*tmp
```

If env files with secrets are already in git history, that's a separate workstream — rotate secrets first, then scrub history with git-filter-repo or BFG.

Variants for non-standard projects

Monorepo

```
monorepo-root/
├── .github/
│   ├── copilot-instructions.md        ← root router; mentions per-package files exist
│   ├── agents/                        ← orchestrator + cross-package specialists
│   ├── instructions/                  ← cross-package instructions
│   └── prompts/
├── docs/                              ← cross-package docs
│   └── ai-context/
└── packages/
    ├── package-a/
    │   └── AGENTS.md                  ← package-specific notes
    └── package-b/
        └── AGENTS.md
```

(Note: nested .github/ directories don't get loaded the same way — use applyTo: globs in root .github/instructions/ to scope per-package.)

Multi-product in one repo

```
your-product-repo/
├── .github/agents/
│   ├── orchestrator.md
│   ├── product-a-frontend.md
│   ├── product-a-backend.md
│   ├── product-b-frontend.md
│   └── shared-database.md
├── .github/instructions/
│   ├── product-a.instructions.md      (applyTo: "src/products/a/**")
│   └── product-b.instructions.md      (applyTo: "src/products/b/**")
├── docs/ai-context/
│   ├── INDEX.md
│   ├── product-a-experience.md
│   └── product-b-experience.md
```

Mobile + web shared

```
your-app/
├── apps/
│   ├── web/                    ← Next.js / Vite / etc.
│   └── mobile/                 ← React Native / Flutter / native
├── packages/                   ← shared libs
└── .github/agents/
    ├── web-ui.md               ← only edits apps/web/
    ├── mobile-ui.md            ← only edits apps/mobile/
    ├── shared-libs.md          ← only edits packages/
    └── backend-api.md          ← stays separate if hosted elsewhere
```

Organization-level configuration

If you have GitHub Copilot Business or Enterprise:

```
your-org/
├── .github-private/            ← special org-level repo
│   └── agents/
│       └── company-wide-agent.md      ← available across all org repos
└── (regular repos as above)
```

Organization custom instructions are configured in the GitHub UI at organization settings — not in files.

ewpage

08 — Common Pitfalls

GitHub Copilot has its own customization quirks layered on top of generic multi-agent pitfalls. Read this before bootstrapping.

Pitfall 1: Putting the orchestrator's persona in .github/copilot-instructions.md

.github/copilot-instructions.md is auto-loaded into EVERY Copilot interaction — including inline completions and code review. Putting the orchestrator's full delegation prose there means: - Inline completions get slower (the persona loads on every keystroke) - Code review gets polluted with "delegate this to backend-api" instructions - Every plain Chat session forces the orchestrator pattern on simple tasks

Right answer: Keep .github/copilot-instructions.md short (under 200 lines). It's a router. The orchestrator persona lives in .github/agents/<project>-orchestrator.md and only loads when explicitly selected.

Pitfall 2: Code review reads only first 4,000 chars

Copilot code review reads only the **first 4,000 characters** of .github/copilot-instructions.md and each .github/instructions/<NAME>.instructions.md.

If you put critical-for-code-review rules at the bottom of a long instruction file, code review won't see them.

Right answer: - Front-load code-review-relevant rules in each instruction file - Or use excludeAgent: "code-review" for instructions that aren't review-relevant (so you can use the full file body for Chat without confusing code review) - If code review really needs detailed guidance, split into multiple smaller instruction files

Pitfall 3: Forgetting applyTo: makes the file useless

An instruction file with no applyTo: field in its frontmatter doesn't get auto-loaded by

anything — it's just an orphan markdown file.

Right answer: Every .github/instructions/*.instructions.md file MUST have a non-empty applyTo: glob in frontmatter. If you have rules with no clear path scope, they belong in .github/copilot-instructions.md (the auto-loaded router) instead.

Pitfall 4: tools: allowlist is your strongest defense — don't omit it

If you don't declare tools: in an agent's frontmatter, the agent inherits ALL tools available in the current Copilot session — including edit tools, terminal tools, and any installed MCP servers.

For REVIEW-ONLY agents (legal-compliance, security-privacy, architecture-review), this is a hidden disaster. The agent can write to your repo even though its instructions say it can't.

Right answer: Always declare tools: explicitly. For REVIEW-ONLY agents, list ONLY read-side tools. Verify by checking your IDE's agent dropdown — the agent should not be able to invoke edit tools.

Pitfall 5: Cloud Agent vs IDE Chat agent — different contexts

The same .github/agents/<NAME>.md file works in both, BUT: - The cloud agent runs in true context isolation (fresh environment per task) - The IDE chat agent runs within the active Chat session's context — it can see prior messages

If your specialist's body assumes the cloud agent's isolation (e.g. "you have no prior context, start fresh"), it'll behave oddly in IDE chat where prior context exists.

Right answer: Write specialist bodies that work in BOTH contexts: - Don't assume isolation; the agent might inherit Chat context - Don't assume Chat context; the agent might run cleanly in cloud - Use explicit handoff schema (handoff_id, previous_specialist, etc.) so the agent isn't relying on implicit conversation memory

Pitfall 6: Prompt files are public preview — features may shift

.github/prompts/*.prompt.md is in **public preview** as of late 2025 / early 2026. Frontmatter fields and invocation behavior may change in future Copilot releases.

Right answer: Stick to documented frontmatter fields (description, agent). Avoid IDE-specific extensions you can't verify. When prompt files leave preview, check the release notes for any breaking changes.

Pitfall 7: .github/chatmodes/*.chatmode.md is the older format

Custom chat modes (.chatmode.md) predate custom agents (.agent.md). .agent.md is the newer/preferred format with more metadata fields (mcp-servers, target, user-invocable, disable-model-invocation).

Right answer: For new projects, use .github/agents/<NAME>.md. Don't create new .chatmode.md files unless you specifically need a chat-mode-only feature.

Pitfall 8: Treating documentation enforcement as runtime enforcement

The handoff schema, the universal evidence rule, the failure_condition observation, the soft hop limits — these are **documentation discipline**. They work because agents follow their instructions. They do NOT physically prevent a misbehaving agent from emitting a malformed handoff.

What IS runtime-enforced (by the Copilot harness): - tools: allowlists per agent - mcp-servers: per-agent MCP scoping - applyTo: globs in instruction files (auto-loading conditional on path match) - target: field (environment scoping for vscode vs github-copilot) - disable-model-invocation and user-invocable (agent visibility/routing) - Body length cap (30,000 chars)

What is NOT runtime-enforced: - The handoff YAML block being present - The evidence rule being followed - Refusal of vague delegations - Hop limits ("3rd same-specialist delegation → escalate") - Definition of Done completeness

Right answer: Be honest about which layer enforces what. Documentation

discipline catches ~95% of value at ~5% of complexity. If you need hard enforcement of a documentation rule, you'd need a custom CI check, a webhook, or wait for Copilot to expose pre-tool-use hooks.

Pitfall 9: Cross-agent invocation has no allowlist

Unlike some other AI tools, Copilot doesn't expose a per-agent allowlist for which other agents the orchestrator can invoke. The orchestrator's body says "I delegate to backend-api / frontend-ui / etc." — but if it tries to invoke a non-existent or unintended agent name, there's no harness-level rejection.

Right answer: - Document the allowed specialist list explicitly in the orchestrator's body - If the orchestrator tries to invoke an unknown specialist, treat it as a documentation bug - For hard enforcement (Business / Enterprise tiers), use organization-level agent definitions to control which agents exist at all

Pitfall 10: MCP server auth lives in ${{ secrets.* }}

Copilot MCP server config supports environment variable substitution from secrets:

```
mcp-servers:
  some-mcp:
    type: 'local'
    command: 'some-tool'
    env:
      API_KEY: ${{ secrets.SOME_MCP_API_KEY }}
```

Don't hardcode API keys in .github/agents/*.md files. Use the secrets reference syntax. Configure secrets at the user, repository, or organization level depending on scope.

Pitfall 11: AGENTS.md is a "lowest common denominator" file

AGENTS.md (or CLAUDE.md, GEMINI.md) is recognized by Copilot as a fallback/cross-AI metadata file. Per the docs, it's "currently not supported by all Copilot features."

Right answer: Don't put critical Copilot-specific config in AGENTS.md. Use .github/copilot-instructions.md for Copilot-specific routing. Use AGENTS.md only for shared cross-AI metadata that you want all AI tools (Copilot + Claude + Gemini) to read.

Pitfall 12: Inline completions only see the first ~few thousand chars of context

Inline completions are latency-sensitive. The model context for inline completions is intentionally smaller than Chat. Don't expect inline completions to know about complex orchestration patterns — they only see: - The current file's open content - Some surrounding files (depending on IDE configuration) - .github/copilot-instructions.md (if present, but truncated for latency) - Path-matching .github/instructions/*.instructions.md (truncated)

Right answer: Optimize .github/copilot-instructions.md for inline completions: lead with the most important constraints in the first 500 chars. The orchestration / agent routing details belong in agent files (which inline completions don't load anyway).

Pitfall 13: Ignoring .env* history

If env files with secrets are already in git history, removing them from the working tree doesn't remove the secrets from history.

Right answer: 1. Rotate every credential that was in any leaked env file. 2. Use git-filter-repo or BFG Repo-Cleaner to scrub history. 3. Force-push the scrubbed history (coordinate with the team). 4. Add broad gitignore patterns: .env*.bak*, .env*staging_tmp, .env*tmp.

Pitfall 14: Letting docs/ rot without a librarian

Without active maintenance, docs/ accumulates: sprint reports, audit reports referenced by no one, outdated decisions, etc. After a year, engineers can't tell which are authoritative.

Right answer: Have a context-librarian specialist whose job is exactly this. Schedule a quarterly cleanup pass. Move stale material to docs/_archive/<YYYY-MM>/.

Pitfall 15: Reorganizing .github/ config without checking IDE behavior

If you rename .github/agents/foo.md → .github/agents/bar.md mid-task, IDE Copilot Chat may not pick up the change until you reload the IDE. The cloud agent picks up new agent files on the next assignment.

Right answer: - Reload your IDE after structural changes to .github/ - Verify with the agent dropdown — the renamed agent should appear - For team-wide changes, communicate so others reload too - Avoid renames mid-PR; do structural changes in their own PR

Pitfall 16: One huge instructions file instead of scoped files

Putting all rules into .github/copilot-instructions.md defeats the purpose of .github/instructions/. Repository-wide instructions load on every interaction; path-globbed instructions load only when relevant.

Right answer: Refactor a long .github/copilot-instructions.md into: - Keep router content (golden rules + workflow + cross-links) in .github/copilot-instructions.md (target < 200 lines) - Move per-area gotchas to .github/instructions/<domain>.instructions.md (with applyTo: globs) - Move long architecture explanations to docs/<UPPERCASE>.md - Move sprint history to docs/_archive/

Pitfall 17: Bootstrap on a repo with existing Copilot config

If you ran VS Code's /init previously, or your team has been adding to .github/copilot-instructions.md for months, BOOTSTRAP-PROMPT.md must NOT silently overwrite that work.

Right answer: the prompt now includes mandatory pre-flight checks at the top — snapshot to .github-pre-bootstrap-backup/, naming-collision detection, applyTo: glob conflict detection, drift detection on existing instructions, and a decision gate that STOPS if any pre-flight raised a <NEEDS USER CONFIRMATION> flag.

Specific risks: - Existing .github/copilot-instructions.md with team rules → Pre-flight 4 (drift detection) flags stale content; Step 11 (merge step) shows a 3-pane diff before writing. - Existing .github/agents/<name>.md with same name as a proposed specialist → Pre-flight 2 (naming collision check) STOPS for explicit user decision per file. - Existing .github/instructions/ with overlapping applyTo: → Pre-flight 3 detects glob overlap; both files would load creating contradictory rules. - Existing .github/chatmodes/ heavily used → Pre-flight 5 surfaces the parallel system; user decides migrate vs coexist.

The pre-flight workflow is non-optional — even on apparent greenfield projects, run it. Cost is 30 seconds; benefit is never silently destroying team work.

Pitfall 18: Forgetting that custom agents are per-repository

.github/agents/<NAME>.md is per-repo. If you have multiple repos and want shared agents, you have two options: - Copy agent files into each repo (drift risk) - For Business/Enterprise: use .github-private/agents/<NAME>.md at the org level

Personal-tier Copilot doesn't support cross-repo agents.

Right answer: For multi-repo orgs on Business/Enterprise, define shared infrastructure in .github-private/. For personal/single-repo, just keep agents in the one .github/.

ewpage

09 — Runbook: Bootstrap a New Project

Step-by-step guide for adopting this framework on a real codebase using GitHub Copilot. Plan for ~2-4 hours of focused work.

Prerequisites

- **GitHub Copilot subscription** (Individual / Business / Enterprise) and signed in
- **Copilot extension** installed in your IDE (VS Code / Visual Studio / JetBrains / Eclipse / Xcode)

- **Recent extension version** that supports custom agents (.github/agents/*.md)
- **Git** installed; **GitHub CLI** (gh) optional for PR creation
- **Read access** to this framework at ~/Desktop/github-copilot-orchestration-framework/
- **2-4 hours** of focused time

Phase 0 — Decide if you need this framework (10 min)

Skip this framework if: - Your project is a solo prototype - Your project has fewer than 4 distinct domain areas - Your team uses Copilot only for inline completions (not Chat or Cloud Agent) - Your project is < 5,000 lines of code

Use this framework if: - Your project has multiple roles - Multiple data systems are in play - Real compliance/security/payments concerns exist - A team uses Copilot daily on the same codebase - You've experienced cascading hallucinations or context drift

Phase 1 — Inventory your project (30 min)

Open your IDE in the project root. Open Copilot Chat.

Paste prompts/INVENTORY-PROMPT.md (from this framework). Copilot will: - Scan your codebase structure - Propose a list of specialists based on your domain boundaries - Surface clutter that should be archived - Identify your protected branches - Identify your tech stack and which MCP servers will be useful

You answer: confirm/adjust the proposed specialist list.

Common gotcha: Copilot may propose 10-15 specialists for a medium project. That's too many. Aim for 5-8.

Phase 2 — Bootstrap the framework files (45 min)

In the same Copilot Chat session, paste prompts/BOOTSTRAP-PROMPT.md. Reference this framework's path so Copilot can read templates:

```
The framework lives at /Users/<you>/Desktop/github-copilot-orchestration-framework/.
Read templates from there. Customize for this project: <project-name>.
```

Copilot will: 1. Create .github/agents/<project>-orchestrator.md from the orchestrator template 2. Create one .github/agents/<specialist>.md per specialist 3. Create .github/instructions/<domain>.instructions.md files (with applyTo: globs) 4. Create .github/prompts/investigate-bug.prompt.md and .github/prompts/build-feature.prompt.md from skill templates 5. Create docs/ai-context/HANDOFF_SCHEMA.md 6. Create docs/ai-context/INDEX.md with task → docs map 7. Create docs/ai-context/ORCHESTRATION_SPOONFEEDER.md 8. Create skeleton docs/ai-context/<area>-experience.md files 9. Create .github/copilot-instructions.md (the router) 10. Create docs/_archive/README.md 11. Update .gitignore

Verify each file before saving.

Phase 3 — Add your first instructions (30 min)

The hardest part of this framework is writing useful instruction files. They take real domain knowledge.

In the Copilot Chat session, ask:

```
Based on the codebase scan, propose 3-5 instruction files for .github/instructions/.
For each, list:
  - applyTo: glob list (verify the globs match real files in this project)
  - 2-3 hard rules with WHY + HOW TO APPLY
  - excludeAgent: if relevant

Do not propose any rule that's just style preference — only invariants where
breaking them caused or could cause a real production issue. Cite file:line
where the gotcha currently lives.

Wait for my approval before creating any files.
```

Common instruction files to start with: - backend-api.instructions.md (or your equivalent server-side concern) - frontend-ui.instructions.md (or mobile-ui.instructions.md) - database.instructions.md - auth-security.instructions.md - testing.instructions.md

Aim for **3-5 rules per file** at first.

Phase 4 — Add 1-2 starter prompt files (20 min)

Most projects benefit from at least these: - .github/prompts/investigate-bug.prompt.md -

.github/prompts/build-feature.prompt.md

Ask Copilot:

```
Create .github/prompts/investigate-bug.prompt.md and
.github/prompts/build-feature.prompt.md based on
templates/prompt.md.template, customized for this project's
tech stack and roles.

These should be invokable via /investigate-bug and /build-feature in Chat.
```

You can add more prompt files later (qa-flow, compliance-review, audit-pipeline, context-refactor).

Phase 5 — Verify (15 min)

Run these checks:

```
# 1. Files in expected locations
ls .github/agents/
ls .github/instructions/
ls .github/prompts/
ls docs/ai-context/

# 2. Doc-link sweep — find broken refs
grep -rohE "(docs|\.github)/[A-Za-z0-9_./-]+\.md" .github/copilot-instructions.md docs/ .github/agent

# 3. Build still passes (whatever your build command is)
npm run build  # or: pytest, go build, cargo build, etc.
```

In your IDE: 4. **Reload the IDE** so Copilot picks up the new .github/ files. 5. Open Copilot Chat → click the agent dropdown → confirm your specialists appear in the list. 6. Type / in Chat → confirm your prompt files appear in the picker.

Fix any breakage before moving on. Common issues: - Agent file YAML invalid (it won't appear in the dropdown) - applyTo: glob points to a path that doesn't match anything (instruction won't load) - Markdown link in .github/copilot-instructions.md to a file that doesn't exist (typo)

Phase 6 — Run a real task through the orchestrator (30 min)

Pick a small real task — ideally a bug fix or a small feature.

Open Copilot Chat. Select your <project>-orchestrator agent from the agent dropdown (or type @<project>-orchestrator at the start of your message).

Verify the active agent shown in Chat is your orchestrator.

Give it the real task. Watch carefully: - Does the orchestrator emit the outbound handoff: YAML block when it proposes to delegate? - When you accept the delegation (or invoke the specialist), does the specialist return the inbound return: YAML block? - Does the orchestrator catch any rejected_claims correctly? - Does verification (tests, build) actually run?

If anything is off → tighten the relevant agent's body. Common fixes: - Orchestrator skipping the YAML block → tighten its "outbound discipline" section - Specialist accepting vague delegations → tighten its "incoming handoff validation" section - Specialist not running tests → add explicit "tests_run MUST be non-empty" to its Definition of Done

Phase 7 — Document for your team (20 min)

Add to your project's existing onboarding docs: - "We use GitHub Copilot with a custom orchestration setup. See docs/ai-context/ORCHESTRATION_SPOONFEEDER.md." - "Always pick the right invocation mode — see docs/06-INVOCATION-MODES.md (or the spoonfeeder's invocation modes section)." - "Don't push to main without project-owner approval."

Optional: announce in your team chat with a 30-second demo of @<project>-orchestrator handling a multi-step task.

Phase 8 — Add hardening as needed (optional, after 2 weeks)

After running real tasks for a couple weeks, decide if you need:

Tighter `tools:` allowlists

If a specialist is doing things outside its domain (e.g. running terminal commands when it shouldn't), tighten its `tools:` array.

Organization-level custom instructions (Business / Enterprise)

If you have multiple repos using this framework, define shared cross-repo conventions at the organization level (UI at github.com org settings + `.github-private/agents/` for shared agents).

CI check for handoff schema compliance

If specialists keep emitting malformed return blocks, add a CI script that grep's PRs for the schema and warns when missing. (Custom CI, not a Copilot feature.)

`excludeAgent: "code-review"` on noisy instructions

If code review is over-applying instructions (e.g. flagging style preferences), add `excludeAgent: "code-review"` to those instruction files.

Phase 9 — Quarterly maintenance

Every 3 months, dedicate a session to: 1. Run the `context-refactor` prompt (or have the `context-librarian` specialist do a sweep) 2. Move stale dated reports to `docs/_archive/<YYYY-MM>/` 3. Reconcile any instruction conflicts 4. Update orientation maps for areas where reality has drifted 5. Verify all agents still appear in the IDE Chat dropdown after dependency / IDE updates

Common time-sinks to avoid

- **Don't try to make every instruction file perfect on day 1.** Ship 5; add 1 per month as production teaches you.
- **Don't over-specialize.** 6-8 specialists is plenty.
- **Don't write canonical docs from scratch in this framework.** Use what you already have.
- **Don't manually merge agent files when adding a rule.** Always edit the instruction file.
- **Don't forget to reload the IDE after `.github/` changes.** New agents won't appear until reload.

When to stop and ask

- If specialists don't appear in the IDE Chat dropdown → YAML frontmatter syntax error
- If instruction files don't auto-load → check `applyTo:` glob matches the file you're editing
- If specialist outputs don't include the return YAML block → the specialist's body needs the "Return schema" section
- If the orchestrator keeps delegating in a loop → check `failure_condition` is articulated
- If a specialist refuses a delegation → check the orchestrator's outbound block has all required fields

End state

You have: - A `<project>-orchestrator` and 4-10 specialists in `.github/agents/` - 3-5 path-globbed instruction files in `.github/instructions/` - 2-4 prompt files (slash commands) in `.github/prompts/` - Three-tier docs (orientation maps + canonical refs + archive) - A team that knows how to pick invocation modes - Auditable handoffs for every cross-domain task

Time invested: 2-4 hours setup + ~30 min/quarter maintenance.

Time saved: every cascading-hallucination incident you DIDN'T have. Every cross-domain task that got the right specialists without you thinking about it. Every new engineer who could navigate the codebase without 2 weeks of pairing.

ewpage

Part II — Prompts

The three prompts to paste into Copilot Chat at different points in the lifecycle.

INVENTORY PROMPT — GitHub Copilot version

Paste this into Copilot Chat at the root of the project you want to bootstrap. Adjust <framework path> to match your install location.

I want to adopt the GitHub Copilot Orchestration Framework on this project. Before we generate any files, do a read-only inventory.

The framework lives at <framework path> (default: /Users/<you>/Desktop/github-copilot-orchestration-framework/). Read docs/01-PRINCIPLES.md, docs/02-ARCHITECTURE.md, and docs/03-AGENTS-GUIDE.md from there before answering. Do not modify any files in this project yet.

⚠ Two universal rules for this entire pass

1. **Evidence-first.** Every claim you make must cite a real file path, command output, or filename. The phrase "I infer" or "appears to be" without a specific file:line citation is not acceptable.
2. **Ask, don't guess.** If a fact cannot be confirmed by reading a specific file or running a specific command, mark it <NEEDS USER CONFIRMATION: <one-line question for me>> and EXPLAIN what you tried before giving up. Examples:
   - <NEEDS USER CONFIRMATION: Is this project named "acme" or "acme-store"? package.json says "@acme/store-monorepo" — slug is ambiguous.>
   - <NEEDS USER CONFIRMATION: Are EU users in scope? No GDPR-related env vars or consent UI found, but README mentions "international expansion".>

Surface ambiguity. Never invent a fact to fill a gap.

Discovery commands — run these in order

Before writing anything, execute this discovery sequence and capture the results in your context. If a command/file is not present, note it as missing and move on.

Step A — Top-level inventory

```
pwd
ls -la
find . -maxdepth 1 -type f | sort
find . -maxdepth 2 -type d | sort
git remote -v
git branch -a
git log --oneline -5
```

Step B — Project name + tech stack manifest discovery

Read whichever of these exist (in this order); stop after the first one with a clear name:

File	Looks for
package.json	name, description, dependencies, scripts (Node/JS/TS)
pyproject.toml	[project] name, dependencies (Python)
setup.py	name=, install_requires= (legacy Python)
Cargo.toml	[package] name, [dependencies] (Rust)
go.mod	module <path>, require (Go)
Gemfile + *.gemspec	gem name, deps (Ruby)
composer.json	name, require (PHP)
pubspec.yaml	name:, dependencies: (Flutter/Dart)
*.podspec / Podfile	iOS native dependencies
build.gradle / build.gradle.kts	Android / Kotlin
*.csproj	.NET project name
package.swift	Swift Package Manager
mix.exs	Elixir

If MULTIPLE manifests exist (monorepo), list all and ask: <NEEDS USER CONFIRMATION: Which manifest is the canonical project name source?>

Step C — Framework + service detection

Read these files if present:

File	Tells you
next.config.js / next.config.ts	Next.js
nuxt.config.ts	Nuxt
vite.config.ts / vite.config.js	Vite
svelte.config.js	SvelteKit
astro.config.mjs	Astro
remix.config.js	Remix
gatsby-config.js	Gatsby
angular.json	Angular
vue.config.js	Vue CLI
app.json / metro.config.js	React Native / Expo
pubspec.yaml (lib: flutter)	Flutter
tsconfig.json	TypeScript target/strict mode
prisma/schema.prisma	Prisma ORM
drizzle.config.ts	Drizzle ORM
*.sql files in migrations/ or db/	Raw SQL migrations
Dockerfile / docker-compose.yml	Container setup
vercel.json	Vercel deploy
netlify.toml	Netlify deploy
wrangler.toml	Cloudflare Workers
serverless.yml	Serverless framework
fly.toml	Fly.io
firebase.json	Firebase
terraform/*.tf / cdk.json	IaC
.github/workflows/*.yml	GitHub Actions CI/CD
.gitlab-ci.yml	GitLab CI
.circleci/config.yml	CircleCI
playwright.config.ts	Playwright E2E tests
cypress.config.ts	Cypress E2E tests
vitest.config.ts / jest.config.js	Unit test runner
pytest.ini / tox.ini	Python test config

Step D — External service detection (.env / dependencies)

Read .env.example (or .env.sample / .env.template). For each env var, infer the service:

Env var pattern	External service
STRIPE_*	Stripe (payments)
RESEND_*, SENDGRID_*, MAILGUN_*, POSTMARK_*	Transactional email
TWILIO_*	Twilio (SMS / voice)
OPENAI_*, ANTHROPIC_*, GEMINI_*	LLM provider
SUPABASE_*	Supabase
FIREBASE_*	Firebase
CLERK_*, NEXTAUTH_*, AUTH0_*, WORKOS_*	Auth provider
POSTHOG_*, MIXPANEL_*, SEGMENT_*, AMPLITUDE_*	Analytics
SENTRY_*	Error monitoring
DATADOG_*, NEW_RELIC_*	APM / observability
CLOUDFLARE_*, R2_*, S3_*, GCS_*	Storage / CDN
ALGOLIA_*, MEILISEARCH_*, ELASTICSEARCH_*	Search
REDIS_URL, UPSTASH_*	Cache / queue
DAILY_*, LIVEKIT_*, AGORA_*, PUSHER_*, ABLY_*	Real-time / video

Cross-check by reading the dependencies section of the manifest from Step B. The two should agree.

Step E — Existing Copilot configuration

```
ls .github/ 2>/dev/null
cat .github/copilot-instructions.md 2>/dev/null | head -50
ls .github/agents/ 2>/dev/null
ls .github/instructions/ 2>/dev/null
ls .github/prompts/ 2>/dev/null
ls .github/chatmodes/ 2>/dev/null
cat AGENTS.md 2>/dev/null | head -50
```

If the project ALREADY has any of these, document what's there. The bootstrap will need to merge with existing config rather than overwriting it.

Step F — Source-tree shape

```
find . -maxdepth 4 -type d \
  -not -path './node_modules/*' \
  -not -path './.git/*' \
  -not -path './.next/*' \
  -not -path './dist/*' \
  -not -path './build/*' \
  -not -path './target/*' \
  -not -path './venv/*' \
  -not -path './__pycache__/*' \
  | sort | head -100
```

For the apparent source root (src/, app/, lib/, internal/, cmd/, etc.):

```
find <source-root> -maxdepth 3 -type d | sort
```

Step G — Documentation discovery

```
find docs -type f -name "*.md" 2>/dev/null | sort
ls docs/ai-context/ 2>/dev/null
find . -maxdepth 1 -type f -name "*.md" | sort   # README, CONTRIBUTING, AGENTS, etc.
```

Step H — Hygiene scan

```
find . -maxdepth 1 -type f -name "_*" -o -name "check-*" -o -name "test-*" 2>/dev/null
find . -maxdepth 1 -type f \( -name "*.png" -o -name "*.jpg" -o -name "*.html" \) 2>/dev/null
find . -maxdepth 2 -type f -name ".env*" 2>/dev/null
git ls-files | grep -E "(\\.env|secret|credential)" | head
```

Step I — Branch and remote check

```
git branch -a | head -20
git remote -v
cat .github/workflows/*.yml 2>/dev/null | grep -E "branches:|push:" | head
```

This identifies protected branches by what CI runs against.

Now produce the structured inventory in 7 sections

Use the discovery results above. Cite SPECIFIC files for every claim.

1. Project identity

- Project name (one phrase) — cite which manifest you read it from
- Project slug (lowercase-hyphenated)
- One-sentence description
- Primary tech stack — list each component with the file that confirmed it
- Deployment target — cite the deploy config file
- CI/CD — cite the workflow file(s)
- Protected branches — list with confirmation source
- Test runners — unit + E2E (cite config files)
- **Existing Copilot config** — what already exists in .github/copilot-instructions.md, .github/agents/, .github/instructions/, .github/prompts/, .github/chatmodes/, AGENTS.md. Bootstrap will need to merge with these, not overwrite.

2. Roles and audiences

List every distinct user type. Infer from route group structure, permission-model files, README. For each role: - Name - One-sentence description - Evidence (file path or doc)

3. Domain boundaries (proposed Copilot agents)

Propose specialists. For each: - name: (lowercase-hyphenated, domain-shaped not technology-shaped) - One-sentence description - Whether it should be REVIEW-ONLY or implementation - The path globs it will primarily edit (must match real directories — verify with ls) - Suggested tools: allowlist (read-only set for REVIEW-ONLY; full set for implementation) - Whether it needs MCP server access (e.g. browser MCP for QA) - Evidence: which signals from Steps C/D/F led you to propose this

Aim for 5-10 specialists. Always include: - An orchestrator named <project-slug>-orchestrator - A qa-functional if user-facing surface (test config exists in Step C) - A release-devops if deploy automation exists - A security-privacy (REVIEW-ONLY) if user data exists - A legal-compliance (REVIEW-ONLY) ONLY if regulatory exposure is confirmed (NOT speculatively)

4. Path-globbed instruction files (proposed)

For each specialist's domain, propose 1-2 instruction files in .github/instructions/. For each: - File name (<NAME>.instructions.md) - applyTo: glob list (verify the globs match real files) - Whether to use excludeAgent: "code-review" (if rules aren't review-relevant) - Top 3-5 hard rules — each WITH source-confirmed evidence (cite file:line)

If you cannot find evidence for a proposed rule: - Either: don't propose it - Or: mark <NEEDS USER CONFIRMATION: I think rule X applies because Y, but I can't find a code example. Is this rule real?>

5. Existing documentation tier

Categorize every file in docs/: - **Canonical** — full-detail, authoritative, currently accurate - **Orientation candidate** — would benefit from being condensed into docs/ai-context/<area>.md - **Archive candidate** — dated reports, sprint snapshots, post-mortems - **Active workflow** — currently-in-progress plans

6. Clutter / hygiene findings

Surface (don't fix yet): - Tracked tool caches (build artifacts, IDE caches) - Orphan scripts at root - Empty stub directories - Loose screenshots / images - Env file backups (.env.local.bak*, .env.staging_tmp) — **flag CRITICAL if tracked in git** - Files with stale paths - Obvious security concerns (env files in git history, hardcoded secrets)

7. Invocation guidance for the team

Propose a one-paragraph snippet for the team's onboarding doc explaining when to use: - Plain Copilot Chat (no agent) - @<project-slug>-orchestrator for cross-domain / production-sensitive work - @<specialist> for narrow single-domain work - Cloud Agent for well-defined, autonomous tasks - /<workflow> slash commands for repeatable workflows

Customize for THIS project's actual specialist list and protected branches.

Output format rules

- Use the section headings above EXACTLY. Numbered 1-7.
- Cite a specific file path or command output for every claim.
- For anything you can't verify, mark <NEEDS USER CONFIRMATION: <specific question>>.
- Do NOT create any files in this pass. Read-only investigation.
- After producing the inventory, end with a section called ## Open questions listing every <NEEDS USER CONFIRMATION> from above as a numbered list.
- Then ask which sections I want to adjust before bootstrapping.

(End of prompt.)

ewpage

BOOTSTRAP PROMPT — GitHub Copilot version

Paste this AFTER you have completed the INVENTORY-PROMPT pass and approved the proposed specialists. Adjust <framework path> to match your install location.

I've reviewed the inventory. Now bootstrap the GitHub Copilot Orchestration Framework on this project. Use the templates from <framework path> (default: /Users/<you>/Desktop/github-copilot-orchestration-framework/templates/). Read those

template files before generating anything.

⚠ PRE-FLIGHT SAFETY CHECKS (run BEFORE creating anything)

This project may already have some Copilot configuration (e.g. from running VS Code's /init, or from prior team work). Bootstrap is designed to coexist with existing config, but ONLY if you detect collisions first. Run these checks in order. Do NOT skip even if the inventory said the project was greenfield.

Pre-flight 1 — Auto-snapshot (mandatory)

Before any write, create a backup of any existing Copilot config:

```
mkdir -p .github-pre-bootstrap-backup
[ -f .github/copilot-instructions.md ] && cp .github/copilot-instructions.md .github-pre-bootstrap-bacl
[ -d .github/agents ] && cp -r .github/agents .github-pre-bootstrap-backup/
[ -d .github/instructions ] && cp -r .github/instructions .github-pre-bootstrap-backup/
[ -d .github/prompts ] && cp -r .github/prompts .github-pre-bootstrap-backup/
[ -d .github/chatmodes ] && cp -r .github/chatmodes .github-pre-bootstrap-backup/
[ -f AGENTS.md ] && cp AGENTS.md .github-pre-bootstrap-backup/
ls -la .github-pre-bootstrap-backup/
```

Report what was backed up. If the backup directory is empty, the project had no prior Copilot config — proceed without further pre-flight concerns.

If the backup directory has content, continue with pre-flight 2-5.

Pre-flight 2 — Naming collision check (per file)

For EACH file you plan to create, check if the destination path already exists:

```
# For each proposed agent
for agent in <project-slug>-orchestrator <specialist-1> <specialist-2> <...>; do
  if [ -f ".github/agents/$agent.md" ]; then
    echo "COLLISION: .github/agents/$agent.md already exists"
  fi
done

# For each proposed instruction file
for inst in <name-1> <name-2> <...>; do
  if [ -f ".github/instructions/$inst.instructions.md" ]; then
    echo "COLLISION: .github/instructions/$inst.instructions.md already exists"
  fi
done

# For each proposed prompt file
for prompt in investigate-bug build-feature <...>; do
  if [ -f ".github/prompts/$prompt.prompt.md" ]; then
    echo "COLLISION: .github/prompts/$prompt.prompt.md already exists"
  fi
done
```

For EACH collision found, STOP and ASK: > <NEEDS USER CONFIRMATION: .github/agents/<name>.md already exists. Options: (a) overwrite (existing content goes to backup), (b) skip this agent, (c) merge content. Which?>

Do NOT silently overwrite ANY collision. If the user picks (c) merge, propose the merged content and get explicit approval before writing.

Pre-flight 3 — applyTo: glob conflict check

For each new instruction file you plan to create, check if existing instruction files have an OVERLAPPING applyTo: glob:

```
# Read all existing instruction file frontmatters
for f in .github/instructions/*.instructions.md; do
  [ -f "$f" ] && echo "=== $f ===" && head -5 "$f" | grep -E "^applyTo:"
done
```

If your proposed applyTo: "src/api/**/*.ts" overlaps with an existing instruction's glob, BOTH instructions will load when editing matching files — leading to potentially contradictory rules. STOP and ASK: > <NEEDS USER CONFIRMATION: My proposed .github/instructions/api.instructions.md (applyTo: "src/api/**/*.ts") overlaps with existing .github/instructions/server.instructions.md (applyTo: "src/api/**,src/server/**"). Options: (a) merge into the existing file, (b) tighten my proposed glob to a non-overlapping subset, (c) accept the overlap (both will load). Which?>

Pre-flight 4 — Drift detection on existing .github/copilot-instructions.md

If .github/copilot-instructions.md exists, READ IT and compare against current project reality:

```
# Re-read current tech stack from manifest
cat package.json 2>/dev/null | head -30
cat Cargo.toml 2>/dev/null | head -30
cat go.mod 2>/dev/null | head -10
cat pyproject.toml 2>/dev/null | head -30

# Compare to what existing copilot-instructions.md says
cat .github/copilot-instructions.md
```

Look for stale claims in the existing file: - Tech stack mentions that don't match current dependencies (e.g. file says "Vue 3" but package.json has React) - Directory paths that don't exist anymore (e.g. file says src/components/ but the project moved to app/components/) - Conventions referencing removed features (e.g. mentions of /api/v1/legacy/ routes that were deleted)

For EACH stale entry found, STOP and ASK: > <NEEDS USER CONFIRMATION: Existing .github/copilot-instructions.md says "<stale claim>" but current reality is "<actual>". Should I drop / update / preserve as-is?>

Do NOT silently drop content. The existing rules may be valuable context the team added intentionally.

Pre-flight 5 — Existing chatmodes / agents differ in style

If .github/chatmodes/ or .github/agents/ already has files, check whether they follow a similar persona/contract style. Two parallel conventions in the same repo confuses the team.

```
ls .github/chatmodes/ 2>/dev/null
ls .github/agents/ 2>/dev/null
```

If existing agents exist with persona contracts that overlap with what you'd create, STOP and ASK: > <NEEDS USER CONFIRMATION: Existing agent .github/agents/code-reviewer.md exists with its own persona. Should the new <project-slug>-orchestrator coordinate WITH it (treat it as another specialist), REPLACE it, or IGNORE it? If you have invested in the existing agent system, replacement risks losing team knowledge.>

Decision gate — STOP if ANY pre-flight check raised an issue

If pre-flight 1-5 raised even one <NEEDS USER CONFIRMATION> flag, STOP and present ALL flags as a numbered list. Wait for the user to answer EVERY flag before proceeding to Step 1 below. Do not partially proceed.

If pre-flight 1-5 raised zero issues (truly greenfield Copilot setup), proceed to Step 1.

What to create (order matters)

Step 1 — Create the directory skeleton

```
.github/agents/
.github/instructions/
.github/prompts/
docs/ai-context/
docs/_archive/
```

(.github/chatmodes/ is OPTIONAL — only create if the project specifically needs the older chat-mode format alongside agents.)

Step 2 — Create docs/ai-context/HANDOFF_SCHEMA.md

Use templates/HANDOFF_SCHEMA.md.template. Substitute: - <PROJECT_NAME> — project's full name from inventory section 1 - <PROJECT_SLUG> — lowercase-hyphenated slug

Customize the worked examples to use realistic file paths from THIS project's codebase. Show me before saving.

Step 3 — Create docs/_archive/README.md

Use templates/archive-README.md.template. No placeholder substitution needed. Show me before saving.

Step 4 — Create docs/ai-context/INDEX.md

Use templates/INDEX.md.template. Populate the routing table from inventory section 3 (specialists) and section 5 (orientation candidates). Show me before saving.

Step 5 — Create the orchestrator agent

.github/agents/<project-slug>-orchestrator.md — use templates/orchestrator-agent.md.template. Substitute: - <PROJECT_NAME> — project's identity - <PROJECT_SLUG> — slug - <PROTECTED_BRANCH> — confirmed protected branch name from inventory - <PROJECT_SPECIFIC_ANTI_PATTERNS> — 3-5 anti-patterns specific to this project

For the tools: field, propose a read-only set:

```
tools: ['codebase', 'search', 'usages', 'problems', 'changes', 'fetch']
```

⚠ Verify the exact tool names supported by the user's IDE/Copilot version. The list above is a common default; some IDEs may use different identifiers. If unsure, ASK the user (<NEEDS USER CONFIRMATION: My proposed tools array uses [codebase, search, usages, ...]. Are these the correct identifiers for your Copilot version?>).

Show me the file before saving.

Step 6 — Create each specialist agent

For each specialist from inventory section 3: - If REVIEW-ONLY: use templates/review-only-agent.md.template - If implementation: use templates/specialist-agent.md.template

Substitute placeholders. The Incoming handoff validation and Return schema (required) sections are IDENTICAL across every specialist — paste verbatim from the template.

For the tools: field: - REVIEW-ONLY agents: read-only tools only (['codebase', 'search', 'usages', 'problems', 'changes', 'fetch'] — adjust per IDE) - Implementation agents: read tools + edit tools (['codebase', 'search', 'usages', 'problems', 'changes', 'edit_files', 'apply_patch', 'runCommands'] — adjust per IDE)

For specialists that need MCP servers (e.g. browser MCP for QA), add the mcp-servers: block per Copilot docs.

Show me each file. Do them in this order so I can check the pattern, then approve subsequent ones in batch: 1. First implementation specialist: full review 2. First REVIEW-ONLY specialist: full review 3. Remaining specialists: batch review (just confirm correct frontmatter + correct cross-links)

Step 7 — Create path-globbed instruction files

For each instruction file from inventory section 4: - Create .github/instructions/<NAME>.instructions.md - Use templates/instructions.md.template - Substitute the actual applyTo: glob and the 3-5 rules

Verify each applyTo: glob matches real files (find . -path '<glob>' — should return matches).

Show me each file before saving.

Step 8 — Create starter prompt files (slash commands)

Create: - .github/prompts/investigate-bug.prompt.md - .github/prompts/build-feature.prompt.md

Both based on templates/prompt.md.template, customized for this project's tech stack and roles.

Optionally create more (/qa-flow, /compliance-review, /context-refactor) if the project needs them.

Show me each file before saving.

Step 9 — Create skeleton orientation maps

For each major domain in inventory section 5, create docs/ai-context/<area>.md with a 50-150 line skeleton: - 2-sentence orientation - "Key file paths" (extracted from codebase scan) - "Top gotchas" (start with placeholders, mark <TODO: fill from real bugs>) - "Cross-links" (link to canonical docs that already exist)

Step 10 — Create docs/ai-context/ORCHESTRATION_SPOONFEEDER.md

Use templates/SPOONFEEDER.md.template. Customize the invocation modes section with the project's specific orchestrator name and specialist list.

Step 11 — Create / update .github/copilot-instructions.md

If the file doesn't exist: create from templates/copilot-instructions.md.template. Substitute placeholders.

If the file EXISTS (verified during pre-flight 1):

1. The original is already snapshotted in .github-pre-bootstrap-backup/copilot-instructions.md.
2. **Read the existing file in full.** Identify which sections are: (a) project-specific rules the team added intentionally, (b) auto-generated /init content that may be stale, (c) generic best-practice content the framework's template covers anyway.
3. Produce a PROPOSED MERGED version that:
   - Preserves ALL section (a) content (project-specific team rules)
   - Drops or updates section (b) content if pre-flight 4 (drift detection) flagged it as stale
   - Replaces section (c) content with the framework's structured router
4. **Show the user a 3-pane diff** before writing:
   - LEFT: existing file (from backup)
   - RIGHT: proposed merged file
   - MIDDLE: a per-section disposition list ("preserved", "updated for drift", "replaced by template router", "dropped — covered by template")
5. Wait for explicit user approval ("yes, write the merged file") before writing. If the user pushes back on any section, revise and re-show the diff.

⚠ **Keep this file UNDER 200 lines** post-merge. Code review reads only the first 4,000 characters; inline completions are latency-sensitive. The file should be a tight router.

⚠ **Front-load the most important rules** in the first 1,500 characters so code review sees them.

If the merged file would exceed 200 lines: propose moving sections to .github/instructions/<NAME>.instructions.md (path-globbed) or docs/ai-context/<area>.md (orientation map) instead of bloating the router. Get user approval per moved section.

Step 12 — Update .gitignore

Append (don't overwrite) these entries:

```
# Bootstrap backup — keep local-only, do not commit
.github-pre-bootstrap-backup/

# Tool caches that should never be committed
test-results/
logs/
coverage/

# IDE caches
.vscode/.history
.idea/workspace.xml

# Vercel/CLI env exports — broader pattern
.env*.bak*
.env*staging_tmp
.env*tmp
```

Adjust based on the project's actual tech stack. The .github-pre-bootstrap-backup/ line is critical — that directory contains the original Copilot config from BEFORE the bootstrap and should NEVER be committed (it would create a confusing parallel set of files in the repo).

Constraints

- **Do not commit anything.** Show me each file. I'll commit at the end.
- **Do not modify existing application code.** Only create/update files in .github/, docs/ai-context/, docs/_archive/, and .gitignore.
- **Pre-flight checks are mandatory** (see top of prompt). Do not skip even on apparent greenfield. If pre-flight raises ANY <NEEDS USER CONFIRMATION> flag, STOP and present all flags before any file write.
- **Snapshot first, write second.** Pre-flight 1 creates .github-pre-bootstrap-backup/ — that backup is your safety net. If you have NOT created the backup, do not proceed to Step 1.
- **Never silently overwrite.** Any pre-existing file at a destination path you'd write requires explicit user approval (per pre-flight 2). "Merge" is not the default — ASK whether to overwrite, skip, or merge.
- **Cite evidence for every gotcha.** If you can't find evidence, mark <NEEDS USER CONFIRMATION> rather than guessing.
- **Verify Copilot tool identifiers.** Before generating agent files with tools: arrays, confirm the tool names are valid for the user's Copilot extension version. ASK if unsure.
- **REVIEW-ONLY specialists must NOT have memory:** if Copilot adds that field — per general AI tooling pitfalls, memory often auto-grants Edit/Write. Currently this isn't a documented Copilot field, but check before adding.
- **Don't put the orchestrator persona in .github/copilot-instructions.md.** That file is auto-loaded for inline completions and code review — would pollute everything.

After all files are created — verify

Run these commands and report results:

```
# 1. Files in expected locations
ls .github/agents/
ls .github/instructions/
ls .github/prompts/
ls docs/ai-context/

# 2. Doc-link sweep — find broken refs
grep -rohE "(docsl\.github)/[A-Za-z0-9_./-]+\.md" .github/copilot-instructions.md docs/ai-context/ .gi

# 3. If existing copilot-instructions.md was merged: diff against backup
diff .github-pre-bootstrap-backup/copilot-instructions.md .github/copilot-instructions.md | head -10(

# 4. If existing agents/instructions/prompts were preserved or merged: diff each
for f in .github-pre-bootstrap-backup/agents/*.md 2>/dev/null; do
  [ -f "$f" ] && [ -f ".github/agents/$(basename "$f")" ] && diff "$f" ".github/agents/$(basename "$f")" |
done

# 5. Build still passes (use whatever the project's build command is)
<project's build command>
```

Then ask the user to: 6. **Reload their IDE** so Copilot picks up the new .github/ files. 7. Open Copilot Chat → click the agent dropdown → confirm specialists appear. 8. Type / in Chat → confirm prompt files appear in the picker. 9. Spot-check a previously-working flow to confirm no regression (e.g. existing slash command still works, existing instruction file still loads on its applyTo: paths).

Report: - Number of agent files created (should match what the inventory proposed) - Number of pre-existing files preserved / merged / overwritten (per user's pre-flight decisions) - Any broken doc links (should be zero) - Build pass/fail - Any <NEEDS USER CONFIRMATION> items still pending - Confirmation that .github-pre-bootstrap-backup/ was created and contains the original files

After verification passes: - I'll review the diff against .github-pre-bootstrap-backup/, commit, and push to a branch + open a PR. - The backup directory should be gitignored (add .github-pre-bootstrap-backup/ to .gitignore if not already). - Once the PR is merged and verified in real use for ~1 week, the backup directory can be deleted locally.

(End of prompt.)

ewpage

REFINEMENT PROMPT — GitHub Copilot version

Use this prompt AFTER you've run 2-3 real tasks through the orchestrator (@<project-slug>-orchestrator in Copilot Chat) and want to harden the setup.

Adjust <framework path> to match your install location.

The orchestration framework has been live for [N] tasks. Now I want a hardening review. Read <framework path>/docs/08-COMMON-PITFALLS.md first.

Review the following

1. Schema compliance

Inspect the most recent 3 orchestrated task transcripts (you can ask me for them or look at recent git history / PR descriptions). Check:

- Did the orchestrator emit the outbound handoff: YAML block on every delegation?
- Did each specialist return the inbound return: YAML block?
- Were failure_condition items articulated meaningfully (not just "task fails")?
- Did any specialist refuse a vague delegation? If yes, what triggered it?
- Were do_not_pass_downstream_without_verification items honored on subsequent hops?

For each violation found, propose a fix: - Tighten the orchestrator's outbound discipline section - Tighten a specialist's incoming validation section - Add a missing instruction file - Add a missing orientation map

2. Tool boundary enforcement

Check each agent file in .github/agents/: - Each agent's tools: field is explicit (no missing field = inheriting everything = bad) - REVIEW-ONLY agents have NO edit tools (no edit_files, apply_patch, runCommands) - The tool identifiers used are valid for the current Copilot extension version - Browser-testing specialists scope MCP via mcp-servers: — not by enumerating individual tools

If a specialist is doing things outside its declared tools: boundary, investigate whether the IDE/extension is still respecting the allowlist correctly. (Bug reports possible — file with GitHub Copilot.)

3. Instruction file usage

For each .github/instructions/<NAME>.instructions.md file: - Is the applyTo: glob still matching real files in the codebase? (find . -path '<glob>') - Are any "hard rules" describing fixed bugs (the constraint no longer exists)? → propose deletion - Did the instruction file actually auto-load during recent tasks? (Specialists should report which instructions applied — if your instructions file isn't appearing in their reports, the glob may be wrong.) - Is the file < 4,000 chars (so code review reads all of it)? If > 4,000 chars, suggest splitting OR adding excludeAgent: "code-review". - Do new instruction files need to be added based on recent production incidents?

4. Specialist scope drift

For each specialist, check the last 3 tasks it handled: - Did it routinely refuse work that "almost" fit its domain? → its scope is too narrow - Did it routinely accept work that the description doesn't really cover? → its scope is too broad - Did it consistently delegate aspects it could have handled inline? → consider broadening - Did it consistently get re-delegated to (orchestrator delegating to it 2-3x in a row)? → soft hop limit triggered; investigate root cause

Propose specific edits to agent definitions, NOT broader restructuring.

5. Doc tier hygiene

Check docs/: - Are there new dated reports / sprint summaries / audit reports at docs/ root that should move to _archive/<YYYY-MM>/? - Are any orientation maps in docs/ai-context/ over 200 lines? (Move detail to canonical doc.) - Is the docs/_archive/README.md current with any new "known link rot" entries?

6. Cloud Agent vs IDE Chat behavior

If the team uses both: - Are agent files written to work in BOTH contexts (no assumptions about isolation)? - Are any target: fields scoping agents unnecessarily to one environment? - Has the cloud agent been assigned tasks that turned out to need back-and-forth (those should have been IDE Chat)?

7. .github/copilot-instructions.md health

- Is the file under 200 lines?
- Is critical-for-code-review content in the first 4,000 characters?
- Has it accumulated debug notes or sprint history that should move out?

8. Hardening recommendations (yes/no per item)

Based on what you've observed, recommend whether to add:

- **excludeAgent: "code-review" on noisy instructions** — if code review is over-applying instructions that aren't review-relevant.
- **Tighter tools: allowlists** — if any specialist is using tools outside its declared scope.
- **Organization-level configuration** (Business / Enterprise) — if you have multiple repos using this framework and want shared agents.
- **AGENTS.md at root** — if the project uses multiple AI tools (Copilot + Claude + Gemini) and would benefit from a shared cross-AI metadata file.
- **CI check for handoff schema compliance** — custom CI script that grep's PRs for the schema and warns when missing.
- **Scheduled context-refactor** — automatic quarterly cleanup pass via a calendar reminder.

For each recommendation, give a one-paragraph justification.

Output format

Produce a structured report:

```
# Refinement Review — <date>

## Tasks reviewed
1. <task 1 description, outcome>
2. <task 2 description, outcome>
3. <task 3 description, outcome>

## Schema compliance
[findings + proposed fixes]

## Tool boundary enforcement
[findings + proposed fixes]

## Instruction file usage
[findings + proposed fixes]

## Specialist scope drift
[findings + proposed fixes]

## Doc tier hygiene
[findings + proposed fixes]

## Cloud Agent vs IDE Chat
[findings + proposed fixes]

## .github/copilot-instructions.md health
[findings + proposed fixes]

## Hardening recommendations
- [ ] excludeAgent: "code-review" on noisy instructions — recommended? Why?
- [ ] Tighter tools: allowlists — recommended for which agents?
- [ ] Organization-level config — recommended? Why?
- [ ] AGENTS.md at root — recommended?
- [ ] CI handoff schema check — recommended?
- [ ] Scheduled context-refactor — recommended?

## Proposed PR
- Files to update (paths)
- Risk assessment per file
- Verification steps
```

Constraints

- **Do not modify any files in this pass.** Read-only review.
- **Wait for my approval per finding.** I'll say which fixes to land vs. defer.
- **Do not propose features GitHub Copilot doesn't currently support.** If you suggest hooks, organization-level features, or new frontmatter fields, verify against the current Copilot docs first.

(End of prompt.)

ewpage

Part III — Templates

ewpage

Template: copilot-instructions.md

```
# <PROJECT_NAME> — GitHub Copilot Router

> **Identity.** <ONE_SENTENCE_PROJECT_DESCRIPTION>. Stack: <TECH_STACK_ONE_LI
>
> **This file is the router, not the manual.** It points to the right docs / agents / instructions per tasl

---

## Golden rules (always)

1. **Never push to `<PROTECTED_BRANCH>` without the project owner's explicit approval.**
2. Always start on `<DEFAULT_DEV_BRANCH>`. Run `<BUILD_COMMAND>` before commit.
3. Don't refactor outside the task. Preserve existing behavior unless asked.
4. For data changes: trace UI → API → DB → tests before touching.
5. <PROJECT_SPECIFIC_GOLDEN_RULE>
6. Don't read every doc. Use the routing map below.
7. **Before editing any file, the relevant `.github/instructions/*.instructions.md` will auto-load if the f
8. **Multi-agent handoffs use a structured schema** — see `docs/ai-context/HANDOFF_SCHEMA

## Mandatory workflow (every non-trivial task)

1. Restate the task.
2. Identify affected roles (<LIST_OF_ROLES>).
3. Read `docs/ai-context/INDEX.md`.
4. Read only the minimum docs flagged for this task (50–150 lines each).
5. Verify that path-matching `.github/instructions/*.instructions.md` auto-loaded.
6. For medium/complex tasks, invoke `@<PROJECT_SLUG>-orchestrator` in Copilot Chat. For u
```

6.5. **Handoff format.** Orchestrator → specialist delegations MUST use the outbound `handoff:`
7. Inspect code before proposing changes.
8. Smallest safe change.
9. Run relevant checks (<TEST_COMMAND>, build, etc.).
10. **Definition of done:** report changed files, tests, risks, follow-ups, AND the list of `.github/ins

---

Context routing — pick by task area

Task touches…	Read this orientation map first
<ROLE_1> dashboard / progress / history	`docs/ai-context/<role-1>-experience.md`
<ROLE_2> dashboard / settings	`docs/ai-context/<role-2>-experience.md`
<FEATURE_AREA_1>	`docs/ai-context/<feature-1>.md`
<FEATURE_AREA_2>	`docs/ai-context/<feature-2>.md`
Database / schema / migrations	`docs/ai-context/database-schema-guide.md`
Auth / sessions / data exposure	`docs/ai-context/security-privacy.md`
Compliance / regulatory	`docs/ai-context/legal-compliance.md`
Tests / E2E	`docs/ai-context/testing-strategy.md`
Build / deploy / cron	`docs/ai-context/release-deployment.md`
Cross-role behavior / acceptance criteria	`docs/ai-context/roles-and-permissions.md`
Repo orientation / what is the product?	`docs/ai-context/product-overview.md`

Agent routing — pick by task class

Task class	Use
Anything medium/complex	`@<PROJECT_SLUG>-orchestrator` (it picks the rest)
<DOMAIN_1>	`@<specialist-1>`
<DOMAIN_2>	`@<specialist-2>`
<DOMAIN_3>	`@<specialist-3>`
Functional QA in browser	`@qa-functional`
Auth / privacy / data exposure	`@security-privacy`
Compliance review (REVIEW-ONLY)	`@legal-compliance`
Build / deploy / cron / env	`@release-devops`
Docs / instructions / agents cleanup	`@context-librarian`

Prompt files (slash commands)

- `/investigate-bug` — bug investigation flow
- `/build-feature` — feature build flow
- `/qa-flow` — QA pass
- `/compliance-review` — compliance review (flagger only, never approver)
- `/context-refactor` — keep `.github/` and `docs/` tidy

Reference docs (canonical — don't duplicate; link to them)

- Multi-agent handoff schema: `docs/ai-context/HANDOFF_SCHEMA.md`
- Architecture: `docs/ARCHITECTURE.md`
- API reference: `docs/API.md`
- Tech stack: `docs/TECH_STACK.md`
- Recent commits: `docs/CHANGELOG.md`
- <ANY_OTHER_CANONICAL_DOCS>

Branches & deploy (universal)

Branch	Environment	URL
`<PRODUCTION_BRANCH>`	production	<PRODUCTION_URL>
`<STAGING_BRANCH>`	staging	<STAGING_URL>

<DEPLOY_MECHANISM_DESCRIPTION>

Local dev

```bash
<DEV_COMMANDS>
```

How to use this Copilot setup

For human-readable usage with copy-paste prompt templates, read **docs/ai-context/ORCHESTRATION_SPOONFEEDER.md**.

ewpage

Template: HANDOFF_SCHEMA.md

```markdown
# <PROJECT_NAME> — Multi-Agent Handoff Schema

> Single source of truth for how the orchestrator delegates to specialists, and how specialists return findings. Bidirectional. Documentation-only — no runtime enforcement (the schema is enforced by agents following their instructions, plus the `tools:` allowlist on each agent).

## Why this exists

<PROJECT_NAME> runs an orchestrator-worker pattern (`.github/agents/<PROJECT_SLUG>-orchestrator.md` + N specialists). Without a structured handoff:

- Orchestrator delegations can be vague ("fix the bug") and the specialist hallucinates the missing context.
- Specialist responses can omit which claims were verified vs. assumed, so the orchestrator passes assumptions to the next specialist as fact.
- Cascading hallucinations compound across hops because nothing forces evidence-to-claim binding at the boundary.

This schema closes both gaps with two YAML blocks and one universal evidence rule.

## Universal evidence rule

> **No evidence, no claim.** If a claim cannot be tied to a file, line, table, column, instruction file, doc, test, or command output, it must be marked as an assumption (`confidence: low` outbound, or moved to `unverified_claims` inbound) and cannot be passed downstream as fact.

This rule applies to BOTH directions of every handoff. It is repeated verbatim in `.github/agents/<PROJECT_SLUG>-orchestrator.md` and in every specialist agent file.

## Outbound: orchestrator → specialist

When the orchestrator `@`-mentions a specialist (or invokes via the cloud agent flow), the prompt body MUST include this YAML block at the top:

```yaml
handoff:
  schema_version: 1
  handoff_id: <short unique id, e.g. h-2026-05-02-a3f>
  from_agent: <PROJECT_SLUG>-orchestrator
  to_agent: <specialist name, e.g. backend-api>
  goal: <one sentence — what done looks like>
  task_class: <bug | feature | review | qa | refactor | audit>
  affected_roles: [<role1>, <role2>]
  claims:
    - claim: <factual statement the orchestrator is committing to>
      evidence: <path/to/file.ts:42 OR instruction:.github/instructions/foo.instructions.md OR doc:docs/ARCHITECTURE.md#section OR table:column OR test:path OR command:output>
      confidence: <high | medium | low>
  verify_before_acting:
    - <claim or assumption the specialist MUST re-verify against source before editing>
  failure_condition: <one sentence — observable evidence that proves the orchestrator's hypothesis or delegation premise is wrong. If the specialist observes this, it MUST stop and return status: claim_rejected (or needs_clarification) instead of editing on a false premise.>
  in_scope:
    - <path or component the specialist may edit>
  out_of_scope:
    - <path or component the specialist must NOT touch — even if "while we're in there">
  instructions_to_read:
    - <.github/instructions/<NAME>.instructions.md path that applies to in-scope files, OR "specialist will discover from applyTo: globs">
  expected_artifacts:
    - <file changed, test added, summary section, etc.>
  hop_context:
    parent_task: <one line — why the orchestrator is delegating this>
    previous_specialist: <agent name or "none">
    previous_findings_summary: <one line — what came back, or "n/a">
```

Field rules (outbound)
```

Field	Required	Notes
schema_version	yes	Currently 1. Bump on breaking changes.
handoff_id	yes	Short unique id. Specialist echoes it in the return so they pair.
from_agent	yes	Always <PROJECT_SLUG>-orchestrator for outbound.
to_agent	yes	The specialist's name field from its .github/agents/<NAME>.md frontmatter.
goal	yes	One sentence.
task_class	yes	One of the six values.
affected_roles	yes	Non-empty list.
claims	yes (may be empty list)	Every entry MUST have evidence.
verify_before_acting	yes if task touches code	Empty allowed only for pure-research handoffs.
failure_condition	yes	One sentence. The inverse of verify_before_acting.
in_scope	yes	Non-empty list.
out_of_scope	yes	Empty list [] is valid. Missing key is not.
instructions_to_read	yes	Use ["specialist will discover from applyTo globs"] if you don't know.
expected_artifacts	yes	Drives the specialist's files_changed / tests_run later.
hop_context	yes	All three sub-fields required. Use none / n/a for first hop.

Inbound: specialist → orchestrator (return schema)

Every specialist response MUST end with this YAML block:

```yaml
return:
  schema_version: 1
  handoff_id: <copy from inbound — pairs the response>
  from_agent: <specialist name>
  to_agent: <PROJECT_SLUG>-orchestrator
  status: <completed | blocked | needs_clarification | claim_rejected>
  verified_claims:
    - claim: <orchestrator claim that was confirmed against source>
      evidence: <path:line or command output the specialist saw>
  rejected_claims:
    - claim: <orchestrator claim that turned out to be wrong>
      evidence: <what the specialist actually found>
  unverified_claims:
    - claim: <orchestrator claim that was not checked>
      reason: <why — out of scope, infra not available, etc.>
  evidence_used:
    - <path:line, instruction file, doc anchor, or command output the specialist relied on>
  files_changed:
    - <path:line range and one-line summary>
  tests_run:
    - <command + result>
  risks:
    - <risk + mitigation or "none">
  recommended_next_agent:
    agent: <specialist name or "none">
    why: <one line>
  do_not_pass_downstream_without_verification:
    - <claim or finding from this specialist that the orchestrator must NOT propagate to the next spe
```

Field rules (inbound)

Field	Required	Notes
schema_version	yes	Must match outbound.
handoff_id	yes	Must echo the inbound id verbatim.
from_agent / to_agent	yes	Self-explanatory.
status	yes	completed only if all verify_before_acting items checked.
verified_claims	yes (may be [])	Empty list is valid.
rejected_claims	yes (may be [])	Non-empty implies status: claim_rejected or blocked.
unverified_claims	yes (may be [])	Anything you accepted from inbound without re-checking goes here.
evidence_used	yes	Real paths/commands. "I read the codebase" is not evidence.
files_changed	yes (may be [])	Empty list valid for review/audit.
tests_run	yes (may be [])	Empty list valid for pure-doc changes.
risks	yes	Use ["none"] if there are genuinely none.
recommended_next_agent	yes	agent: "none" if work is complete.
do_not_pass_downstream_without_verification	yes	Cascading-hallucination defense.

Refusal (specialist response when handoff is invalid)

If the inbound handoff fails the field rules, the specialist responds with ONLY this block, does no other work, and waits for the orchestrator to re-issue:

```
return:
  schema_version: 1
  handoff_id: <copy from inbound, or "missing" if absent>
  from_agent: <specialist name>
  to_agent: <PROJECT_SLUG>-orchestrator
  status: needs_clarification
  reason: handoff_schema_invalid
  missing_or_invalid_fields:
    - field: <field name>
      problem: <what's wrong — missing, vague, no evidence, etc.>
  required_action:
    - <what the orchestrator must add or fix before re-issuing>
```

Worked examples

Example 1 —

Outbound:

```yaml
handoff:
  schema_version: 1
  handoff_id: h-2026-XX-XX-bug-001
  from_agent: <PROJECT_SLUG>-orchestrator
  to_agent: <YOUR_BACKEND_SPECIALIST>
  goal: <REPLACE WITH A REAL BUG INVESTIGATION GOAL FROM YOUR PROJECT>
  task_class: bug
  affected_roles: [<your-role>]
  claims:
    - claim: <REAL CLAIM WITH EVIDENCE FROM YOUR CODEBASE>
      evidence: <real path:line>
      confidence: high
  verify_before_acting:
    - <real verification step>
  failure_condition: <real falsification condition>
  in_scope:
    - <real path>
  out_of_scope:
    - <real adjacent path that should NOT be touched>
  instructions_to_read:
    - <real instructions path>
  expected_artifacts:
    - <real expected outcome>
  hop_context:
    parent_task: <real triggering context>
    previous_specialist: none
    previous_findings_summary: n/a
```

Return:

```yaml
return:
  schema_version: 1
  handoff_id: h-2026-XX-XX-bug-001
  from_agent: <YOUR_BACKEND_SPECIALIST>
  to_agent: <PROJECT_SLUG>-orchestrator
  status: completed
  verified_claims:
    - claim: <claim from inbound>
      evidence: <verification evidence>
  rejected_claims: []
  unverified_claims: []
  evidence_used:
    - <real path:line>
  files_changed:
    - <real file:line range and summary>
  tests_run:
    - <real command + result>
  risks:
    - <real risk or "none">
  recommended_next_agent:
    agent: qa-functional
    why: <real reason>
  do_not_pass_downstream_without_verification: []
```

Example 2 —

[Pattern as above; customize for your stack and roles.]

Example 3 —

[Pattern as above; customize for your REVIEW-ONLY specialist.]

How agents reference this file

- .github/agents/<PROJECT_SLUG>-orchestrator.md requires the outbound block on every @<specialist> invocation and consumes the inbound block before merging.
- Each specialist agent file under .github/agents/ (excluding orchestrator) requires validating the inbound block, refusing if invalid, and emitting the return block on every response.
- .github/copilot-instructions.md golden rule references this doc.
- docs/ai-context/INDEX.md routes “Multi-agent handoff format” questions here.

Versioning

- schema_version: 1 is the current version. Bump only on breaking changes.
- Additive changes (new optional fields) keep version 1.
- When bumping, update HANDOFF_SCHEMA.md first, then every agent file in the same PR.

Cross-links

- .github/agents/<PROJECT_SLUG>-orchestrator.md — the issuer
- docs/ai-context/ORCHESTRATION_SPOONFEEDER.md — human-facing usage guide
- docs/ai-context/INDEX.md — task routing

ewpage

Template: INDEX.md

```markdown
# <PROJECT_NAME> — AI Context Index

> Master router. Map every common task type to the **minimum** docs and agents needed. Read this file early in any non-trivial task.

---

## How to use this index

1. Restate the task in plain words.
2. Skim "Task type → Read this" below.
3. Read ONLY the listed docs (they are 50–150 lines each).
4. If the task is medium/complex, hand it to `@<PROJECT_SLUG>-orchestrator` (it picks the rest).
5. **Path-matching `.github/instructions/*.instructions.md` files auto-load when you edit matching files. Verify they loaded.**

---

## Task type → Read this

| Task type | docs/ai-context/ to read | .github/instructions/ that may apply | Agent (optional) |
|---|---|---|---|
| <ROLE_1> dashboard / progress / history | `<role-1>-experience.md` | `<frontend>.instructions.md`, `<backend>.instructions.md` | `@<frontend-specialist>`, `@<backend-specialist>` |
| <ROLE_2> dashboard / billing / settings | `<role-2>-experience.md` | `<frontend>.instructions.md`, `<backend>.instructions.md` | `@<frontend-specialist>`, `@<backend-specialist>` |
| <FEATURE_AREA_1> | `<feature-1>.md` | `<feature-1>.instructions.md` | `@<feature-1-specialist>` |
| <FEATURE_AREA_2> | `<feature-2>.md` | `<feature-2>.instructions.md` | `@<feature-2-specialist>` |
| Database / schema / migrations | `database-schema-guide.md` | `<database>.instructions.md` | `@<database-specialist>` |
| Auth / sessions / data exposure | `security-privacy.md` | `<security>.instructions.md` | `@security-privacy` |
| Compliance review / regulatory | `legal-compliance.md`, `security-privacy.md` | `<security>.instructions.md` | `@legal-compliance`, `@security-privacy` |
| Tests / E2E | `testing-strategy.md` | `<testing>.instructions.md` | `@qa-functional` |
| Build / deploy / cron / env | `release-deployment.md` | `<deploy>.instructions.md` | `@release-devops` |
| Cross-role behavior / acceptance criteria | `roles-and-permissions.md` + role-specific docs | varies | `@product-flow` |
| Docs / agents / instructions cleanup | (this file + `ORCHESTRATION_SPOONFEEDER.md`) | n/a | `@context-librarian` |
| Multi-agent handoff format | `HANDOFF_SCHEMA.md` | n/a | `@<PROJECT_SLUG>-orchestrator` (issuer) + every specialist (receiver) |

---

## Worked routing examples

(Replace these with examples specific to your project.)

### Example 1 — "<COMMON_BUG_PATTERN_FOR_YOUR_PROJECT>"

1. Read `<orientation-map-1>.md`.
2. When you start editing matching files, `.github/instructions/<rule>.instructions.md` will auto-load.
3. Hand to `@<PROJECT_SLUG>-orchestrator` if scope grows; else go directly to `@<specialist-name>`.

### Example 2 — "<COMMON_FEATURE_REQUEST_FOR_YOUR_PROJECT>"

1. Read `<orientation-map-1>.md` + `<orientation-map-2>.md`.
2. Use prompt file: `/build-feature` in Copilot Chat.
3. Agent: `@product-flow` (acceptance criteria) → topic specialists.

### Example 3 — "<COMMON_AUDIT_PATTERN_FOR_YOUR_PROJECT>"

1. Read `<orientation-map>.md`.
2. Instructions auto-load when reviewing matching paths.
3. Use prompt file: `/compliance-review`.
4. Agent: `@<audit-specialist-name>`.

---

## Reference docs (canonical — don't duplicate; link to them)

- Multi-agent handoff schema: `docs/ai-context/HANDOFF_SCHEMA.md`
- Architecture: `docs/ARCHITECTURE.md`
- API reference: `docs/API.md`
- Tech stack: `docs/TECH_STACK.md`
- Product requirements: `docs/PRODUCT_REQUIREMENTS.md`
- <PROJECT_SPECIFIC_CANONICAL_DOCS>

---

## Pre-refactor source (optional)

If you migrated from a single huge `.github/copilot-instructions.md`, the original is preserved verbatim at `docs/ai-context/LEGACY_BACKUP.md`. The migration ledger at `docs/ai-context/MIGRATION_LEDGER.md` tracks every section/gotcha through the multi-wave migration.
```

ewpage

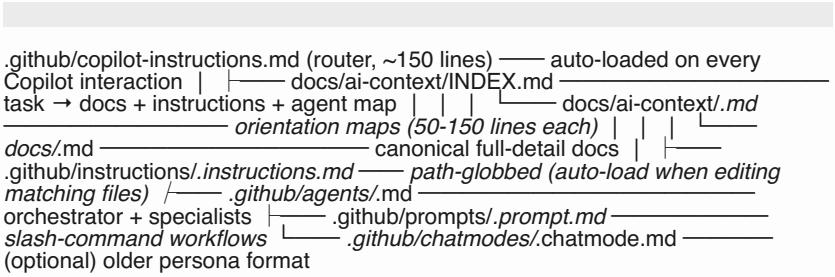
Template: SPOONFEEDER.md

<PROJECT_NAME> — Copilot Orchestration Spoonfeeder

> Practical, copy-paste-friendly guide for the project owner and team. Explains how the GitHub Co

---

The mental model in one picture



The big idea. Tiny `.github/copilot-instructions.md` as router. Every task picks only the docs/agents/instructions it needs. Specialist agents do focused work. The orchestrator dispatches.

What is what

Layer	Where	What it does
Router	`.github/copilot-instructions.md`	Auto-loaded by Copilot on every interaction in this repo. Tiny. Points to everything else.
Routing index	`docs/ai-context/INDEX.md`	Task type → minimum docs + instructions + agent.
Spoonfeeder	`docs/ai-context/ORCHESTRATION_SPOONFEEDER.md`	This file — for humans.
Handoff schema	`docs/ai-context/HANDOFF_SCHEMA.md`	Bidirectional schema for orchestrator ↔ specialist.
Orientation maps	`docs/ai-context/<area>.md`	50–150 lines each. Tool-agnostic.
Canonical docs	`docs/<UPPERCASE>.md`	Untouched. Full detail.
Path-globbed instructions	`.github/instructions/<domain>.instructions.md`	Auto-loaded by Copilot when editing files matching `applyTo:` glob.
Specialist agents	`.github/agents/<name>.md`	One purpose each. Bounded by "I CAN" / "I CANNOT" lists + `tools:` allowlist.
Orchestrator	`.github/agents/<PROJECT_SLUG>-orchestrator.md`	Manager agent. Picks specialists per task.
Prompt files (slash commands)	`.github/prompts/<workflow>.prompt.md`	Repeatable workflows. Invoke via `/<workflow>` in Chat.

Invocation modes — which Copilot surface to use

Mode 1 — Inline completions (autocomplete)

Just type. Copilot autocompletes. Loads only `.github/copilot-instructions.md` + path-matching `.github/instructions/`. No agents, no slash commands.

Use for: typing assistance.

Mode 2 — Plain Chat (no agent selected)

Open Copilot Chat (Ctrl+Cmd+I in VS Code). Type a question.

Use for:
- Quick questions about code
- Single-line copy / typo fixes
- Pure read tasks

Mode 3 — Specialist Chat (`@<specialist>`)

In Copilot Chat, select a specialist from the agent dropdown OR type `@<specialist-name>` at the start of your message.

Use for:
- Narrow single-domain work
- e.g. `@frontend-ui` for UI work
- e.g. `@backend-api` for API work
- e.g. `@qa-functional` for QA pass
- e.g. `@legal-compliance` for review-only session

The agent's `tools:` allowlist physically restricts what it can do.

Mode 4 — Orchestrator Chat (`@<PROJECT_SLUG>-orchestrator`)

Select the orchestrator agent from the dropdown OR type `@<PROJECT_SLUG>-orchestrator` at the start of your message.

Use for:
- Anything spanning more than one role
- Anything spanning more than one feature area
- Data integrity, payments, security/privacy, compliance work

- Data integrity, payments, security/privacy, compliance work
- Bug investigations where root cause may cross layers
- Feature builds where acceptance criteria need to be written before implementation
- Pre-release QA across multiple roles
- Anything where the scope is ambiguous

Mode 5 — Cloud Agent (autonomous)

Assign a Copilot agent to a GitHub issue or PR. The cloud agent runs autonomously on github.com infrastructure.

Use for: well-defined, self-contained tasks.

Mode 6 — Slash commands (`/<workflow>`)

Type `/` in Chat. Pick from the prompt file picker, or type the name: `/<workflow-name>`.

Use for: invoking a documented workflow.

Why we don't make the orchestrator the "always-on" persona

Putting the orchestrator's persona in `.github/copilot-instructions.md` would auto-load it for inline completions and code review — pollution. Keep the persona in `.github/agents/<PROJECT_SLUG>-orchestrator.md` and invoke explicitly via `@<PROJECT_SLUG>-orchestrator`.

Required reading before any non-trivial task

1. `.github/copilot-instructions.md` (auto-loaded — already done by Copilot).
2. `docs/ai-context/INDEX.md` — pick the right docs.
3. The 1–2 orientation maps the INDEX points to.
4. Verify the matching `.github/instructions/*.instructions.md` files auto-loaded for the paths you intend to edit.

That's it. Stop adding more unless the task genuinely needs more.

Copy-paste prompt templates

A. General orchestration

@-orchestrator

Restate the task, identify affected roles, read only the minimum docs from docs/ai-context/INDEX.md, choose the right specialist agents, merge findings into a plan, and do not implement until the plan is clear.

Task:

B. Bug investigation

@-orchestrator

Use /investigate-bug. Investigate this bug. Focus on root cause, impacted roles, data correctness, smallest safe fix, and tests. Verify the matching .github/instructions/*.instructions.md auto-loaded for any files you edit.

Bug: Repro steps: <STEPS or “unknown”>

C. Feature build

@-orchestrator

Use /build-feature. Plan this feature. Identify affected roles, acceptance criteria, frontend/backend/data/security impact, and tests before any implementation.

Feature:

D. Functional QA

@qa-functional

Use /qa-flow. Test this flow. Focus only on functional bugs and incorrect data. Do not give generic improvement suggestions unless they block functionality.

Flow: Roles to cover:

E. Compliance review

@-orchestrator

Use /compliance-review. Coordinate @legal-compliance and @security-privacy. Review this feature for relevant regulations. Do not claim final advice. Produce a reviewer-checklist.

Feature:

F. Release readiness

@release-devops

Review release readiness for the change below. Check build, tests, env vars, staging vs production, cron jobs, rollback plan, and risks.

Change: Target: staging / production

G. Context cleanup

@context-librarian

Tidy the docs/ai-context/, .github/instructions/, .github/agents/, .github/prompts/ surface. Consolidate duplicates, flag conflicts (do not silently resolve), keep .github/copilot-instructions.md small.

Scope:

Definition of done — every task

A task is complete only when the agent reports:

1. What changed (files touched).
2. Why it changed.
3. **Which `.github/instructions/*.instructions.md` files applied to your edits.**
4. Roles affected.
5. Tests / checks run.
6. Risks remaining.
7. Whether security/privacy review is needed.
8. Whether legal/compliance review is needed.
9. Whether docs/context need updates.

If any of these is missing, the task is not done.

Anti-patterns

- **Don't put the orchestrator persona in `.github/copilot-instructions.md`.** It auto-loads everywhere — pollutes inline completions and code review.
- **Don't write rules without `applyTo:`** in `.github/instructions/`. Without the glob, Copilot has no signal to load them.
- **Don't omit `tools:`** on a specialist. Defaulting to "all tools" defeats defense-in-depth.
- **Don't use every agent for every task.** Pick the minimum.
- **Don't broadly refactor while fixing a bug.** Make the smallest safe change.
- **Don't claim compliance is solved without qualified-reviewer sign-off.**
- **Don't put debug notes in `.github/copilot-instructions.md`.** That file should be a tight router.
- **Don't follow this plan blindly.** Re-read the current repo state.

How instructions actually load (important)

`.github/instructions/<NAME>.instructions.md` files are **auto-loaded by Copilot when you edit a file whose path matches the `applyTo:` glob in the frontmatter**. You don't have to invoke them manually.

But: **verify they loaded.** A typo in `applyTo:` means the instruction silently doesn't load. The orchestrator's Definition of Done explicitly requires the list of instruction files that applied. If you see a task summary with no instructions listed, the agent skipped that step — push back.

Cloud Agent vs IDE Chat — when to pick which

Task profile	Pick
Interactive, iterative, ambiguous scope	IDE Chat
Cross-domain, production-sensitive	IDE Chat with `@<PROJECT_SLUG>-orchestrator`
Narrow, single-domain	IDE Chat with `@<specialist>`
Well-defined, self-contained, junior-level	Cloud Agent (assign GitHub issue)
Need to step away while it works	Cloud Agent
Sensitive code that should stay in IDE	IDE Chat

Reminder for Copilot when starting a session

Re-read .github/copilot-instructions.md and docs/ai-context/INDEX.md. Identify task type → pick the minimum docs + agent. Verify .github/instructions/*.instructions.md auto-loaded for paths you'll edit. Use a specialist agent via @ only when useful. Smallest safe change. Definition of done includes instruction files referenced.

ewpage

Template: archive-README.md

docs/_archive/ — Frozen historical material

Stale audit reports, dated sprint/phase documents, one-off verification artifacts, and superseded pl

Why this exists

`docs/` had drifted to N entries mixing canonical references (ARCHITECTURE.md, API.md, etc.) v

Layout

```
_archive/ ├─── 2026-03/ — dated reports from March 2026 ├─── 2026-04/ — dated reports from April 2026 ├─── audit-reports/ — third-party audit notes ├─── html/ — generated HTML reports ├─── screenshots/ — historical UI captures └─── README.md — this file
```

(Adjust the subdirectories to match what you actually have.)

Rules

- **Do NOT link from active docs into _archive/.** Active documentation (`.github/copilot-instructions.md`, `docs/ai-context/*`, `.github/agents/`, `.github/instructions/`, `.github/prompts/`) must reference only canonical sources at `docs/` root or `docs/ai-context/`. Linking to archive content would resurrect drift.
- **Do NOT delete archive content.** History matters for audits and post-mortems. If something is genuinely stale beyond reference value, propose deletion in a PR — do not silently drop it.
- **Do NOT move content out of archive back to `docs/` root.** Archived content is frozen at the moment it was archived. If a topic is still relevant, write a fresh canonical doc that supersedes the archived one.
- **Date subdirectories are append-only.** When adding new archived material, use the year-month folder matching when the content was originally written.

Known link rot (intentional)

If you have a frozen pre-refactor `LEGACY_BACKUP.md` snapshot of the original `.github/copilot-instructions.md`, it may contain links to paths like `docs/SPRINT_*.md` that now resolve to `docs/_archive/<YYYY-MM>/...`. **Do NOT update those links** — the legacy backup represents a point-in-time snapshot. To find the actual files, look in `_archive/<YYYY-MM>/` with the same filename.

When to archive new material

A doc belongs in `_archive/` when ALL of these are true:
- It was written for a specific point-in-time event (sprint, audit, migration, incident)
- The event is complete
- No active instruction file, agent, or canonical doc cites it
- Its information is either superseded by a canonical doc or only useful as historical context

When in doubt, ask: would a new engineer joining the project benefit from reading this file as part of normal onboarding? If yes → keep at `docs/` root. If no → archive.

ewpage

Template: orchestrator-agent.md

```
---
name: <PROJECT_SLUG>-orchestrator
description: Main task dispatcher for <PROJECT_NAME>. Use for any medium/complex task. P
tools: ['codebase', 'search', 'usages', 'problems', 'changes', 'fetch']
target: github-copilot
user-invocable: true
disable-model-invocation: false
---

# <PROJECT_NAME> — Orchestrator Agent

The orchestrator is the manager. It reads the task, classifies it, picks the minimum set of docs/instr

## When to use

- Any task touching more than one role.
- Any task spanning more than one feature area.
- Any task involving data integrity, payments, security/privacy, or compliance.
- Anything where the scope is ambiguous.
- Bug investigations where root cause may cross layers.
- Feature builds where acceptance criteria need to be written before implementation.

## When NOT to use

- Single-line copy fixes.
- Obvious typo fixes.
- Pure read/answer questions ("what does X do?").
- Single-file bug fixes where the rule + specialist agent are obvious from the task description.

## Required reading (every task)

1. `.github/copilot-instructions.md` — auto-loaded; just confirm you've absorbed it.
2. `docs/ai-context/INDEX.md` — pick the orientation maps that match the task.
3. The 1–3 orientation maps the INDEX points to (50–150 lines each).
4. **Path-matching `.github/instructions/*.instructions.md` files auto-load when editing matching file
```

Optional (only when the task warrants):
- `docs/ARCHITECTURE.md`, `docs/API.md`, etc. for canonical detail.

Routing examples

Task class	Specialists
Bug investigation	`@qa-functional` for repro → topic specialist for root cause → `@qa-functio
New feature	`@product-flow` (acceptance criteria) → topic specialists in parallel → `@qa-func
Data integrity	`@<database-specialist>` + `@security-privacy`.
Compliance review	`@legal-compliance` (review-only) + `@security-privacy`.
Release / deploy	`@release-devops` + `@qa-functional`.

(Customize for your project's specialist names.)

Handoff format (required — both directions)

Every delegation from this orchestrator (via `@<specialist-name>` mention or by spawning a spe

Universal evidence rule. No evidence, no claim. If a claim cannot be tied to a file, line, table, cc

Outbound discipline (when delegating):
- Every `claim` MUST cite real evidence (file:line, instruction file path, doc anchor, table:column, te
- `verify_before_acting` MUST list at least one item if the task touches code (the specialist re-ver
- `failure_condition` MUST state, in one sentence, what observation would prove your hypothesis
- `out_of_scope` MUST be filled — empty list `[]` is a valid answer if the task is genuinely contain
- `hop_context.previous_specialist` MUST be set if this is a re-delegation. The specialist uses it
- `handoff_id` is short and unique per delegation — the specialist echoes it on return so the pair c

Inbound discipline (when consuming a return):
- Read `verified_claims` and `rejected_claims` BEFORE merging the specialist's work. A `rejec
- `unverified_claims` and `do_not_pass_downstream_without_verification` items MUST be e
- `status: needs_clarification` with `reason: handoff_schema_invalid` means the outbound wa

Hop limit (soft). If you find yourself delegating to the same specialist a third time on the same `

I CAN

- Restate, classify, and route any task.
- Read all docs, all instruction files, all code (read-only by tool allowlist).
- Coordinate specialists by sending them focused, self-contained `@<specialist>` mentions with t
- Merge specialist findings into a single plan with concrete file paths and line numbers.
- Require QA before claiming a task done.
- Escalate legal/privacy/payment/security risks back to the project owner.
- Suggest adding new gotchas to the appropriate `.github/instructions/*.instructions.md` after a

I CANNOT

- Implement code without a clear plan (always specialists do that, with edit tools they have).
- Use every agent for every task — minimum-set only.
- Skip the instruction-reading directive before edits.
- Issue a delegation without the outbound `handoff:` schema block.
- Hide unverified assumptions inside `goal` instead of declaring them as `claims` with `confidenc
- Omit `failure_condition` — every delegation must articulate what would falsify it. If you cannot r
- Propagate a specialist's `unverified_claims` or `do_not_pass_downstream_without_verifica
- Approve production deploys (project owner only).
- Bypass authentication / authorization / data-validation gates.
- Push to `<PROTECTED_BRANCH>`.

Definition of Done — every orchestrated task

The orchestrator's final summary MUST include:

1. **Files changed** (paths, ideally with `path/to/file.ts:42` line refs).
2. **Why each change was needed.**
3. **Instruction files that applied** (the `.github/instructions/*.instructions.md` paths that auto-lo
4. **Roles affected.**
5. **Tests / checks run.**
6. **Risks remaining.**
7. **Whether security/privacy review is needed** (yes/no + reason).
8. **Whether legal/compliance review is needed** (yes/no + reason).
9. **Whether docs/context need updates.**

If any of these is missing, the task is not done. Push back on specialists that don't include them.

Anti-patterns to refuse

<**PROJECT_SPECIFIC_ANTI_PATTERNS** — replace with **3-5** anti-patterns specific to your proje

- "Just refactor while you're in there" — refuse; smallest safe change.
- "Skip QA, the diff is small" — refuse; QA is part of the Definition of Done.
- "<**Project-specific** gate>" — refuse; <**reason**>.

Cross-links

`docs/ai-context/HANDOFF_SCHEMA.md` — bidirectional handoff schema (required reading f

- `docs/ai-context/HANDOFF_SCHEMA.md` — bidirectional handoff schema (required reading t
- `docs/ai-context/ORCHESTRATION_SPOONFEEDER.md` — the human-facing usage guide.
- `docs/ai-context/INDEX.md` — task → docs + instructions + agent map.
- `.github/prompts/investigate-bug.prompt.md` — bug workflow (slash command `/investigate
- `.github/prompts/build-feature.prompt.md` — feature workflow (slash command `/build-featu

ewpage

Template: specialist-agent.md

```
---
name: <SPECIALIST_NAME>
description: <ONE_SENTENCE_DESCRIPTION_OF_DOMAIN>. Coordinates with @<RELATEI
tools: ['codebase', 'search', 'usages', 'problems', 'changes', 'edit_files', 'apply_patch', 'runCommar
target: github-copilot
user-invocable: true
disable-model-invocation: false
---

# <SPECIALIST_DISPLAY_NAME> Agent

<2_SENTENCE_DESCRIPTION_OF_WHAT_THIS_AGENT_OWNS>

## When to use

- <BULLET_LIST_OF_TASK_TYPES_THIS_AGENT_OWNS>

## When NOT to use

- <BULLET_LIST_OF_TASK_TYPES_THAT_BELONG_TO_OTHER_AGENTS>
- e.g. "General DB schema work — use @database specialist"
- e.g. "UI styling — use @frontend-ui specialist"
- e.g. "Compliance review — use @legal-compliance specialist"

## Required reading

1. `docs/ai-context/INDEX.md` — task → docs map (always check first to confirm scope).
2. `docs/ai-context/<RELEVANT_ORIENTATION_MAP>.md` — orientation for this domain.
3. **`.github/instructions/<RELEVANT_INSTRUCTIONS>.instructions.md`** auto-loads when editi
4. <ADDITIONAL_INSTRUCTIONS_OR_DOCS_PER_DOMAIN>

## Incoming handoff validation

Before doing any work, validate the orchestrator's delegation against the schema in `docs/ai-con

REFUSE the delegation (respond with the rejection template, do nothing else) if any of these are t
- The YAML `handoff:` block is missing entirely.
- `schema_version`, `handoff_id`, `from_agent`, or `to_agent` are missing.
- `goal` is vague ("fix the bug", "improve the page", "make it work").
- Any `claim` lacks `evidence` (a path:line, instruction file, doc anchor, table:column, test, or comr
- `verify_before_acting` is empty AND the task touches code.
- `failure_condition` is missing or empty.
- `out_of_scope` is missing (empty list `[]` is allowed; missing key is not).
- `affected_roles` is missing or empty.

**Universal evidence rule.** No evidence, no claim. If a claim cannot be tied to a file, line, table, cc

For every item in `verify_before_acting`, re-check it against source before editing. If the orchestr

**Failure-condition observation rule.** If at any point during the work you observe the inbound `fai

## Return schema (required)

Every response back to the orchestrator MUST end with the `return:` YAML block defined in `doc

Use `do_not_pass_downstream_without_verification` to flag any finding the orchestrator must

## I CAN

- <BULLET_LIST_OF_PERMITTED_ACTIONS>
- e.g. "Read and edit any code in src/<this-domain>/."
- e.g. "Run unit tests for this domain locally."
- e.g. "Author migrations." (if applicable)

## I CANNOT

- <BULLET_LIST_OF_PROHIBITIONS>
- e.g. "Edit code outside this specialist's domain — escalate to the right specialist."
- e.g. "Apply migrations to production without project-owner approval."
- "Push to `<PROTECTED_BRANCH>` — project-owner approval only."

## Definition of Done

1. Files changed (paths).
2. **Instruction files that applied** — list `.github/instructions/*.instructions.md` files that auto-l
3. <DOMAIN_SPECIFIC_DONE_ITEM> (e.g. "Tests added for new helpers")
4. <DOMAIN_SPECIFIC_DONE_ITEM> (e.g. "Build passes")
5. <DOMAIN_SPECIFIC_DONE_ITEM>

## Cross-links

- `.github/instructions/<related-instructions>.instructions.md` — full rule body.
- `docs/ai-context/<orientation-map>.md` — orientation.
- `<canonical-doc>` — full reference detail.
```

ewpage

Template: review-only-agent.md

```
---
name: <SPECIALIST_NAME>
description: REVIEW-ONLY. Flags <DOMAIN_LIST> risks. Produces reviewer-checklist content.
tools: ['codebase', 'search', 'usages', 'problems', 'changes', 'fetch']
target: github-copilot
user-invocable: true
disable-model-invocation: false
---

# <SPECIALIST_DISPLAY_NAME> Agent

> **Disclaimer.** This agent is a **flagger**, not an approver. Output is for the project owner's qual

> **REVIEW-ONLY contract enforced by `tools:` allowlist.** This agent has NO edit/write tools — it

## When to use

- <BULLET_LIST_OF_REVIEW_TASKS>
- e.g. "Pre-launch gate for any feature touching <regulated-domain>."
- e.g. "Quarterly review of <specific-process>."
- e.g. "Risk assessment for new third-party integrations."

## When NOT to use

- Any task expecting an authoritative <reviewer-type> opinion.
- Code implementation (delegate to topic specialists).
- Any task where the project owner says "this is a non-<review-type> review."

## Required reading

1. `docs/ai-context/INDEX.md` — task → docs map (always check first to confirm scope).
2. `docs/ai-context/<RELEVANT_ORIENTATION_MAP>.md` — orientation for this domain.
3. **`.github/instructions/<RELEVANT_INSTRUCTIONS>.instructions.md`** if any code path is bei
4. <ADDITIONAL_DOCS_OR_CANONICAL_REFS>

## Incoming handoff validation

Before doing any work, validate the orchestrator's delegation against the schema in `docs/ai-con

REFUSE the delegation (respond with the rejection template, do nothing else) if any of these are t
- The YAML `handoff:` block is missing entirely.
- `schema_version`, `handoff_id`, `from_agent`, or `to_agent` are missing.
- `goal` is vague.
- Any `claim` lacks `evidence`.
- `verify_before_acting` is empty AND the task touches code.
- `failure_condition` is missing or empty.
- `out_of_scope` is missing.
- `affected_roles` is missing or empty.

**Universal evidence rule.** No evidence, no claim. If a claim cannot be tied to a file, line, table, cc

For every item in `verify_before_acting`, re-check it against source. If the orchestrator's claim tur

**Failure-condition observation rule.** If at any point you observe the inbound `failure_condition`

## Return schema (required)

Every response back to the orchestrator MUST end with the `return:` YAML block defined in `doc

## Standing checklist (apply to every review)

<NUMBERED_LIST_OF_REVIEW_QUESTIONS_SPECIFIC_TO_THIS_DOMAIN>

Examples for legal-compliance:
1. Is consent already in place for this data flow?
2. Is data retention bounded? Where is the deletion mechanism?
3. Is the data minimized?
4. Is the data portable?
5. Is the data deletable on request?
6. Is third-party processor coverage in place?
7. If feature involves AI inference on user data, is the use disclosed in the privacy policy?
8. Is consent surface visible BEFORE capture starts?
9. If feature crosses borders, does the privacy policy cover the transfer?
10. Are there marketplace / employment / agency clauses to preserve?

Examples for security-privacy:
1. Is there an authentication check?
2. Is there an authorization check (correct role, correct ownership)?
3. Is the input validated and sanitized?
4. Are secrets handled correctly (env vars, never in code, never in logs)?
5. Is the output redacted of PII / sensitive fields?
6. Are rate limits in place?
7. Is the dependency tree free of known CVEs?
```

I CAN

- Read code, docs, configurations.
- Flag risks against <**regulations** / standards / threat models>.
- Produce reviewer-checklist content.
- Recommend changes (the IMPLEMENTATION goes to other specialists).
- Flag missing artifacts.

I CANNOT

- Provide final <**type-of-advice**>.
- Sign off on <**approval-type**>.
- Modify code (the `*tools:*` allowlist physically prevents this).
- Edit canonical policy/terms/spec docs without explicit project-owner approval.
- Approve production launches.

Definition of Done

1. **Risks identified** — severity-ranked (blocker / high / medium / low).
2. **Reviewer checklist** — numbered list of specific questions for the qualified reviewer.
3. **Suggested changes** — what should change before launch.
4. **Ship-pending-reviewer flag** — yes/no (NOT approval — just whether the team should hold fc
5. **Instruction files referenced.**

Cross-links

- `*.github/instructions/<related-instructions>.instructions.md*`
- `*docs/ai-context/<related-orientation-map>.md*`
- <**canonical-policy-or-spec-doc**>

ewpage

Template: instructions.md

```
---
applyTo: "<GLOB_PATTERN_1>,<GLOB_PATTERN_2>"
excludeAgent: "code-review"
---

# <Domain Display Name> Instructions

> **What this is.** Auto-loaded by GitHub Copilot when editing any file matching the `applyTo:` gl
>
> **`excludeAgent: "code-review"`** is OPTIONAL. Use it if these instructions shouldn't apply durir

## Hard rules

### <Rule title — short imperative sentence>

**Why:** <ONE_SENTENCE_REASON — past production incident or hard invariant>

**How to apply:** <2-4 SENTENCES — what to do, what helper to use, what to check. Cite the he

### <Next rule title>

**Why:** <reason>

**How to apply:** <how>

<ADD_AS_MANY_RULES_AS_NEEDED — but split into a separate file if this one passes ~150 l

## What NOT to do

- <BULLET_LIST_OF_ANTI_PATTERNS>
- e.g. "Don't <tempting wrong thing> — <one-line reason>"

## Cross-links

- `docs/ai-context/<related-orientation-map>.md` — orientation for this domain.
- `docs/<RELATED_CANONICAL_DOC>.md` — full reference.
- `.github/instructions/<related-instructions>.instructions.md` — related path-globbed instruct

---

## Notes for the instruction author

(Delete this section before committing.)

A good instruction file:
- Has a real `applyTo:` glob that matches files actually being edited
- Has a real "Why" for each rule (past incident or hard invariant — not aspirational)
- Has a concrete "How" for each rule (the helper to use, the check to run, the pattern to follow)
- Each rule is ≤ 4 sentences (move long detail to canonical doc)
- Front-loads code-review-relevant rules (code review only reads first 4,000 chars)
- Is grouped with related rules under one domain file

A bad instruction file:
- "We should always..." (aspirational; not enforced)
- Pure style preference (lint config does this)
- Contradicts another instruction file (have the context-librarian reconcile)
- Has no `applyTo:` (no signal for Copilot to load it)
- Long rationale paragraph (move to canonical doc)
- > 150 lines / > 3,000 chars (split into multiple files)

When you've shipped 3-5 rules in this file, stop. Add more only when production teaches you new
```

ewpage

Template: prompt.md

```
---
description: <ONE_SENTENCE_DESCRIPTION_OF_WHEN_TO_USE_THIS — appears in the
---

# <Prompt Display Name>

> **Invocation.** In Copilot Chat (VS Code / Visual Studio / JetBrains), type `/<this-filename-with

## When to invoke

- <BULLET_LIST_OF_TRIGGER_CONDITIONS>
- e.g. "Pre-release validation of a new feature."
- e.g. "Bug investigation requires browser repro."
- e.g. "Periodic sanity sweep across roles."

## When NOT to invoke

- <BULLET_LIST_OF_ANTI_TRIGGERS>
- e.g. "Authoring code fixes (delegate to topic specialists)."

## Inputs

- `${input:goal:What is the high-level goal of this task?}`
- `${input:scope:Which paths/components are in scope?}`
- <OTHER_INPUTS_AS_NEEDED>

## Workflow

### 1. <Step name>

<2-4 SENTENCES — what to do, what to produce, what to verify before moving on>

### 2. <Step name>

<...>

### 3. <Step name>

<...>

### 4. <Step name>

<...>

### 5. <Step name>

<...>

(Aim for 5-10 steps. More than 10 is a smell — split into sub-prompts.)

## Definition of Done

1. <ARTIFACT_1 — e.g. "Severity-ranked findings list">
2. <ARTIFACT_2 — e.g. "Files changed (paths)">
3. <ARTIFACT_3 — e.g. "Tests run (commands + results)">
4. <VERIFICATION — e.g. "Build passes locally">
5. **Instruction files that applied** (the `.github/instructions/*.instructions.md` paths that auto-lo

## Cross-links

- `.github/agents/<related-agent>.md` — the specialist this prompt typically pairs with.
- `.github/instructions/<related-instructions>.instructions.md` — the rules this prompt enforce
- `docs/ai-context/<related-orientation>.md` — orientation for the domain.

---

## Notes for the prompt author

(Delete this section before committing.)

A good prompt file:
- Is invoked 3+ times per quarter (otherwise it's not really a recurring workflow)
- Has clear steps with verification between them
- Uses `${input:NAME:HINT}` for any user-supplied values
- Pairs with a specific specialist agent (typically named in cross-links)
- Cites the instructions it enforces

A bad prompt file:
- Vague trigger condition ("when something feels complex")
- Workflow steps that are really just "use Copilot smartly"
- No clear Definition of Done
- Duplicates an agent's body content (the agent already knows its own contract)
- Tries to substitute for an agent (use an agent instead)

If you find yourself writing a prompt that's just a checklist of generic best practices, you don't need
```

ewpage

Template: chatmode.md

```
---
description: <ONE_SENTENCE_PERSONA_DESCRIPTION — appears in the chat mode dropd
tools: ['codebase', 'search', 'usages', 'problems', 'changes']
model: <optional — leave unset to use default>
---

# <Chat Mode Display Name>

> ⚠ **Recommendation:** for new projects, prefer `.github/agents/<NAME>.md` (custom agent
>
> Use this template only if you specifically need a chat-mode-only feature, OR if you have legacy c

<2-3 SENTENCE PERSONA DESCRIPTION — what role does this Copilot mode adopt? When is

## When to use this mode

- <BULLET_LIST_OF_TRIGGER_CONDITIONS>

## When NOT to use this mode

- <BULLET_LIST_OF_ANTI_TRIGGERS>

## Constraints

- <BULLET_LIST_OF_HARD_CONSTRAINTS>
- e.g. "Never edit production migration files."
- e.g. "Always cite file:line for any claim about existing code."

## Style guide

<BULLET_LIST_OF_RESPONSE_STYLE_PREFERENCES>
- e.g. "Prefer concise answers over comprehensive ones."
- e.g. "Use code blocks for any code samples."

## Cross-links

- `docs/ai-context/<orientation-map>.md`
- `.github/instructions/<related-instructions>.instructions.md`

---

## Notes for the chat mode author

(Delete this section before committing.)

Chat modes (`.chatmode.md`) and custom agents (`.agent.md`) overlap heavily. Differences:

| Feature | `.chatmode.md` | `.agent.md` |
|---|---|---|
| Tool allowlist (`tools`) | Yes | Yes |
| Model selection (`model`) | Yes | Yes |
| MCP server scoping (`mcp-servers`) | Limited | Full support |
| Environment scoping (`target`) | No | Yes (`vscode`, `github-copilot`) |
| User-invocable / auto-routing (`user-invocable`, `disable-model-invocation`) | No | Yes |
| Cloud agent support | No | Yes |
| Cross-IDE consistency | Some drift | Better |

Recommended: use `.agent.md` for new work. Keep this template in case you have legacy chat-r
```